

The Blueprint, Part 1: एक User ते कोटी

August 30, 2026

THE BLUEPRINT

Book 5, Part 1: प्रवास: एक User ते कोटी

हे पुस्तक कुणासाठी: Book 1 (The Machine) ने “internet चालतं कसं” सांगितलं. हे पुस्तक पुढचं सांगतं: टिकणारं, कोटी-लोकांचं **software** बांधतात कसं, आणि जगातल्या उत्तम **teams** काम कशा करतात. का महत्वाचं? कारण तुमचं code आता agents लिहितील (Book 4 ने दाखवलं); पण कोणतं बांधायचं, कुठली रचना निवडायची: हा निर्णय अजून तुमचा. आणि तोच खरा फरक: top talent ची ओळख code टाईप करण्याच्या वेगात नाही, **रचनेच्या नजरेत** आहे.

या पुस्तकाचं एकमेव सूत्र

मोठेपणा फुकट नसतो.
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणं.
Trade-off न दिसणं हीच खरी चूक.

उलगडून: software मध्ये “सगळ्यात चांगली” रचना नसते; फक्त “या परिस्थितीला योग्य” रचना असते. वेग हवा तर जागा (किंवा पैसा) विका; खात्री हवी तर वेग विका. जो engineer प्रत्येक निर्णयामागचा **सौदा** बघतो तो top; जो “हे भारी tech आहे म्हणून वापरू” म्हणतो तो नवखा. प्रत्येक chapter शेवटी हे सूत्र परत भेटेल.

आरतीचं किराणा-app: या पुस्तकाचं चालतं उदाहरण

आरती. वय 26. पुण्याजवळच्या तालुक्यातली. तिने एक साधं app बनवलं: गावातल्या किराणा-दुकानांना घरपोच ऑर्डर. सुरुवात: एक दुकान, एक गाव, दिवसाला 10 ऑर्डर, एका जुन्या laptop वर चालणारं. Part 1 मध्ये आपण तिच्यासोबत प्रवास करू: 10 ऑर्डर ते **कोटी** ऑर्डर. आणि हा प्रवास **भिंतींचा** आहे: प्रत्येक टप्प्यावर एक नवी भिंत आदळते (app हळू झालं, database बुडाली, दिवाळीला crash झालं), आणि प्रत्येक भिंत एक नवी संकल्पना शिकवते. आरतीच्या जागी स्वतःला ठेवा: तिचा प्रत्येक पेच तुमचा आहे.

कपाटावर एक नजर

Book 1	The Machine	tech चा पाया
Book 2	The Thinking Machine	AI यंत्र
Book 3	The Land Game	जमीन-धंदा
Book 4	The Harness	AI वर धंदा
Book 5	The Blueprint	<- हे: मोठं software कसं बांधतात

तीन भागांचा नकाशा: तुम्ही इथे आहात

PART 1	प्रवास: एक user ते कोटी	<- तुम्ही इथे server, database, cache, replication, sharding, queue, load balancer, CDN: भिंती एकेक
PART 2	रचनेचे निर्णय	SQL/NoSQL, monolith/microservices, API, search
PART 3	Elite शिस्त + खऱ्या इमारती	git, testing, incident-संस्कृती, WhatsApp/UPI/ Zerodha/Netflix उलगडून

Book 1 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. Server म्हणजे काय? (Book 1)

दुसऱ्यांच्या विनंती ऐकून उत्तर देणारा, सतत चालू असलेला computer.

2. Request-response एका ओळीत?

तुमचा computer विनंती पाठवतो, server उत्तर परत पाठवतो; internet म्हणजे या देवाणघेवाणीचं जाळं.

3. Database का लागते?

माहिती **टिकवायला** आणि पुन्हा शोधायला: वीज गेली तरी राहणारी, रचलेली साठवण.

4. Latency म्हणजे?

विनंती आणि उत्तर यांच्यातला वेळ: अंतर जितकं जास्त, latency तितकी जास्त.

चौघे पक्के? मग आरतीच्या पहिल्या server पासून सुरू करू.

विभाग 1: भिंती एकेक

हा विभाग एक प्रवास आहे, chapters नव्हे. आरतीचं app वाढत जाईल, आणि प्रत्येक टप्प्यावर एक भिंत आदळेल: पहिली विनंती, पहिली database, पहिला हळूपणा, पहिला crash. प्रत्येक भिंत एक संकल्पना आणि एक **सौदा** शिकवेल. **हे तुमच्या धंद्यात कुठे येईल:** तुमचं कुठलंही app/सेवा वाढली की याच भिंती याच क्रमाने येतात: WhatsApp लाही आल्या, UPI लाही. आधी दिसल्या की तयारीने सामोरे जाता; न दिसल्या की crash होऊन शिकता (महाग रस्ता).

Chapter 1.1 [SPINE]: एक Server, एक स्वप्न

आरतीचं app पहिल्या दिवशी एका जुन्या laptop वर चालत होतं. एक दुकान, दहा गिन्हाईकं. आणि तरी त्या छोट्या यंत्रात पुढच्या कोटी-ऑर्डरचं संपूर्ण बीज आहे: कारण एक विनंती जे प्रवास करते, तेच कोटी विनंत्याही करतात: फक्त गर्दी वेगळी. म्हणून आधी **एका विनंतीचं पूर्ण शरीर** बघू.

एका ऑर्डरचा प्रवास. गावातला रमेश app उघडतो, “2 किलो साखर” वर बोट ठेवतो. त्या एका स्पर्शापासून काय-काय घडतं:

1. रमेशचा फोन एक विनंती (request) बनवतो: "साखर, 2 किलो, रमेश, हा पत्ता"
2. ती विनंती internet वरून आरतीच्या server कडे जाते (Book 1: request-response)
3. Server ती वाचतो, विचार करतो (code चालवतो): दुकानात साखर आहे का? भाव किती? रमेशचा पत्ता खरा का?
4. Server database ला विचारतो (माहिती कुठे टिकलीये)
5. उत्तर तयार करतो: "ऑर्डर घेतली, 40 रुपये, अर्ध्या तासात"
6. ते उत्तर परत रमेशच्या फोनवर

सहा पायऱ्या. आणि इथे तीन वेगळी कामं एकाच laptop मध्ये एकत्र नांदत आहेत, हे ओळखा: **ऐकणारा-बोलणारा** (web server: विनंत्या घेतो-उत्तरं देतो), **विचार करणारा** (application: तुमचं code, धंद्याचे नियम), आणि **लक्षात ठेवणारा** (database: माहिती टिकवतो). लहान दुकानाची उपमा: एकच माणूस गल्ल्यावर बसतो, गिन्हाईकाशी बोलतो, हिशेब करतो, **आणि** वही सांभाळतो. चालतं: कारण गिन्हाईकं दहा.

पहिला सौदा (सूत्र सुरू होतं): सोपेपणा वि. वाढ. सगळं एका यंत्रात असणं म्हणजे **सोपं**: एकच जागा, एकच gadget, समजायला सोपं, बांधायला जलद. आरतीने अगदी बरोबर केलं: दहा ऑर्डरसाठी तीन वेगळे computers घेणं म्हणजे मुंगीला हत्तीचा साज. **छोट्या सुरुवातीला “एकत्र सगळं” हीच शहाणी रचना आहे.** पण या सोपेपणाची एक किंमत लपलेली आहे, आणि ती पुढच्या भिंतीवर दिसेल: तो एक laptop म्हणजे **एकच अपयश-बिंदू**: तो बंद पडला की पूर्ण धंदा बंद.

आणि एक महत्त्वाची जोड (Book 1 शी): “server” म्हणजे खास जादूई यंत्र नाही: तो तुमच्यासारखाच computer आहे, फक्त तो **सतत चालू** असतो आणि **दुसऱ्यांच्या विनंत्या ऐकतो**. आरतीचा जुना laptop server आहे कारण त्याने ते **काम** केलं: यंत्रामुळे नाही, भूमिकेमुळे. उद्या तो cloud मधल्या भाड्याच्या यंत्रावर जाईल (Book 1 चं cloud), पण भूमिका तीच राहील.

महाग चूक

”सुरुवातीलाच ‘मोठ्या company सारखं’ बांधू: microservices, अनेक servers, सगळा जामानिमा.” नवशिक्यांची सगळ्यात महाग चूक, आणि तिला नाव आहे: **premature scaling** (वेळेआधी मोठेपणा). दहा ऑर्डरसाठी शंभर-ऑर्डरची गुंतागुंत बांधणारा सहा महिने रचनेतच अडकतो आणि app कधी निघतच नाही. सूत्र उलटं लावा: **आजच्या गर्दीला जेवढं लागतं तेवढंच; भिंत आली की मग पुढंच**. आरती दहा ऑर्डरला एक laptop वापरते: हा भोळेपणा नाही, शहाणपण आहे.

नकाशावर

प्रवासाचा नकाशा (वाढत जाईल):

- 1.1 एक यंत्र, तीन कामं (एक-विचार-लक्षात ठेव); सोपं पण एकच अपयश-बिंदू

सूत्रावर: एका laptop ची रचना सोपेपणा विकत घेते, वाढ विकून. दहा ऑर्डरला हा सौदा फायद्याचा; उद्या तोट्याचा होईल: आणि तेव्हा रचना बदलेल. **कुठलीही रचना कायमची बरोबर नसते; ती त्या-त्या गर्दीला बरोबर असते.**

स्वतः बघा (5 मिनिटं)

तुमच्या रोज वापरातल्या एका app चा विचार करा (WhatsApp, कुठलंही). एक कृती निवडा (message पाठवणं) आणि वरच्या सहा पायऱ्या त्याला लावून बघा: विनंती काय, server काय विचार करत असेल, काय लक्षात ठेवत असेल, उत्तर काय. दिसणारं app साधं; मागचं शरीर हेच.

विचार करा

आरतीला एका रात्री धक्का बसला: laptop चा data चुकून पुसला गेला, आणि आदल्या दिवसाच्या सगळ्या ऑर्डर, दुकानांचे भाव: सगळं गेलं. “माहिती code मध्येच का ठेवत नाही? वेगळी database कशाला?”

कारण code आणि माहिती यांचं आयुष्य वेगळं असतं: code तुम्ही लिहिता-बदलता, माहिती **रोज वाढते आणि टिकावी लागते** (वीज गेली, यंत्र बदललं तरी). दोन्ही एकत्र ठेवणं म्हणजे आरतीची रात्र: पुन्हा-पुन्हा. म्हणून माहितीला स्वतःचं, टिकाऊ, रचलेलं घर लागतं: पहिली खरी भिंत, पुढचा chapter: database चा जन्म.

Chapter 1.2 [SPINE]: पहिली भिंत: Database चा जन्म आणि ACID

आरतीच्या पुसलेल्या-data च्या रात्रीतून पहिली खरी संकल्पना जन्मते. माहितीला वेगळं, टिकाऊ घर हवं: **database**. पण database म्हणजे नुसती “मोठी वही” नाही; ती एका करारावर उभी असते: आणि तो करार समजला की पुढचं अर्ध पुस्तक सोपं होतं.

Database म्हणजे नक्की काय जास्त देते? किराणा-दुकानाची वही आठवा: नुसती यादी. आता बँकेची वही आठवा: तिथे नियम कडक आहेत: प्रत्येक नोंद पूर्ण किंवा मुळीच नाही; एकाच वेळी दोन कारकून एकाच खात्याला हात लावले तर गोंधळ होऊ नये; आणि एकदा नोंदलं की वीज गेली तरी टिकलं पाहिजे. **database ही अशी शिस्तीची वही आहे.** त्या शिस्तीला चार अक्षरांचं नाव: **ACID**. हे चार शब्द पाठ करा; हजारो engineers चं ते समान पाया-ज्ञान आहे:

- A Atomicity (अखंडता): व्यवहार पूर्ण किंवा मुळीच नाही. रमेशचे पैसे कापले पण ऑर्डर नोंदली नाही: असं अर्धवट कधीच नाही. दोन्ही, किंवा काहीच नाही.
- C Consistency (सुसंगती): नियम कधीच मोडत नाहीत. साखर -5 किलो शिल्लक असं अशक्य; नियम database राखते.
- I Isolation (एकांत): दोन ऑर्डर एकाच वेळी आल्या तरी एकमेकांत मिसळत नाहीत; प्रत्येकीला वाटतं ती एकटीच.
- D Durability (टिकाऊपणा): एकदा “झालं” म्हटलं की वीज गेली, यंत्र बंद पडलं तरी नोंद राहते.

आरतीला हे चारही हवे आहेत. साखरेची शिल्लक, रमेशचे पैसे, दुकानाचा हिशेब: इथे “अर्धवट” किंवा “मिसळलेलं” म्हणजे धंदा बुडाला. म्हणून ती एक खरी database वापरते (PostgreSQL: जगात 2026 त developers मध्ये सर्वात लोकप्रिय, Stack Overflow च्या पाहणीत सलग पहिली): नुसती file नाही.

पण इथे सूत्र लपलंय: ACID ची किंमत. ही शिस्त फुकट नाही: प्रत्येक “झालं” म्हणण्याआधी database ला खात्री करावी लागते (खरंच लिहिलं? टिकलं?), एकांत राखायला कुलपं लावावी लागतात. म्हणजे ACID = **खात्री विकत, वेग विकून**. बँकेची वही किराणा-वहीपेक्षा **हळू** असते: कारण ती जास्त **खात्रीशीर** असते. हा सौदा आत्ता (पैसे, शिल्लक) फायद्याचा; पण पुढे आरतीला काही माहिती अशी भेटेल जिथे कडक खात्रीपेक्षा वेग महत्त्वाचा (कोण-किती-वेळा-app-उघडलं सारखी): तिथे हाच सौदा उलटवावा लागेल (Part 2 चा SQL-vs-NoSQL वाद याच फाट्यावर उभा).

खरी जगातली भीषण आठवण (postmortem-चव): database किती नाजूक, हे 2017 त **GitLab** ला महाग कळलं. एका engineer ने रात्री, replication-चा गोंधळ सोडवताना, **चुकीच्या** server वरची database-directory पुसली: ~300 GB. आणि मग कळलं की त्यांच्या **पाच** backup-पद्धतींपैकी एकही

धड चालत नव्हती (backup-files काही bytes च्या, दुसरी रिकामी). शेवटी ~6 तास जुन्या एका snapshot वरून वाचवलं; ~6 तासांचा data (हजारो projects, नोंदी) कायमचा गेला: आणि हे सगळं त्यांनी **थेट YouTube वर live** दुरुस्त केलं. धडा दुहेरी: database हे धंद्याचं हृदय, **आणि** “backup आहे” म्हणणं वेगळं, “backup तपासलेला आहे” म्हणणं वेगळं (Book 4, 4.5 ची तीच शिकवण, इथे रक्ताने). न तपासलेला backup म्हणजे नसलेला backup.

महाग चूक

“माहिती Excel/file मध्येच ठेवू, database कशाला?” छोट्या, न-वाढणाऱ्या, एकट्याच्या यादीला file ठीक. पण जिथे **पैसे, शिल्लक, किंवा एकाच वेळी अनेक लोक** माहिती बदलतात: तिथे file म्हणजे आरतीची पुसली-गेली रात्र आणि त्याहून वाईट: दोन लोकांनी एकाच वेळी लिहिलं तर **गुपचूप** चुकीचा आकडा (isolation नाही). ACID ची गरज “मोठेपणा” वर नाही, **माहितीच्या स्वभावावर** ठरते: पैसा-शिल्लक-नोंद = ACID हवीच.

नकाशावर

- 1.1 एक यंत्र, तीन कामं; एकच अपयश-बिंदू
- 1.2 database = शिस्तीची वही (ACID: अखंड-सुसंगत-एकांत-टिकाऊ); खात्री विकत, वेग विकून

सूत्रावर: ACID हा पहिला मोठा जाणीवपूर्वक सौदा: वेग देऊन खात्री घेतली. Top engineer ला विचारा “ACID हवी का?” तो “कशासाठी?” विचारेला: पैशाला हो, like-count ला कदाचित नाही. सौदा दिसणं = नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनमधल्या दोन गोष्टींचा विचार करा: bank-app मधली शिल्लक, आणि कुठल्याही app मधली “views/likes” ची संख्या. कुठली **अचूक** असलीच पाहिजे (एक रुपयाही चुकून चालणार नाही), आणि कुठली “साधारण बरोबर” चालते? ही रेषा म्हणजे ACID कुठे हवी याची रेषा: तुम्ही ती रोज नकळत अनुभवता.

विचार करा

आरतीचं app वाढलं: 10 वरून 500 ऑर्डर रोज. आणि अचानक app **हळू** झालं: दुकानाची यादी उघडायला 8-10 सेकंद. Database तीच, नियम तेच, मग हळू का? आरती गोंधळली: “माहिती तर आहेच आत, मग शोधायला एवढा वेळ का?”

कारण database माहिती **कशी शोधते** यावर वेग ठरतो, आणि डिफॉल्ट पद्धत म्हणजे “पूर्ण वही पहिल्या पानापासून चाळणं”: 500 नोंदींना ठीक, 5 लाखांना संकट. वही भराभर उघडायला एक जुनी कारकुनी युक्ती लागते: अनुक्रमणिका. पुढची भिंत: index.

Chapter 1.3 [SPINE]: Index: गोदामाची वही

आरतीचं app हळू का झालं? उत्तर एका जुन्या कारकुनी युक्तीत आहे, आणि ती युक्ती समजली की database चा अर्धा वेग-गूढ संपतो.

पूर्ण वही चाळणं वि. अनुक्रमणिका. समजा एका जाड पुस्तकात “साखर” शब्द कुठे आहे शोधायचंय. दोन मार्ग: (1) पहिल्या पानापासून प्रत्येक ओळ वाचत जाणं: 500 पानी पुस्तकाला वेळ लागेल, पण चालेल; 5 लाख पानी पुस्तकाला **आयुष्य** लागेल. (2) मागच्या **अनुक्रमणिकेत** “साखर” बघणं: “पान 213”: थेट तिथे. Database ची डिफॉल्ट पद्धत पहिली आहे (full scan: पूर्ण table चाळणं). आरतीच्या 500 ऑर्डरला ते चालत होतं; 5 लाखांना कोसळलं. **Index म्हणजे database ची अनुक्रमणिका.**

Index कसं काम करतं. आरती database ला सांगते: “रमेशच्या फोन-नंबरवर मी वारंवार शोधते: त्याची अनुक्रमणिका बनव.” Database एक वेगळी, **क्रमाने लावलेली** यादी ठेवते: फोन-नंबर -> “ती नोंद इथे आहे.” आता शोध पूर्ण-चाळणीचा नसतो; अनुक्रमणिकेतला उडी-मारत शोध (5 लाखांत ~19 उड्यांत सापडतं: half-half कापत: Book 6 चा binary search इथे कामाला येईल). 8 सेकंदांचं काम काही milliseconds वर. आरतीचं app पुन्हा भुरकन.

पण सूत्र इथेही हजर: index ची तिहेरी किंमत. फुकट वेग नाहीच: (1) **जागा:** अनुक्रमणिका ही वेगळी यादी: database मोठी होते (पुस्तकाच्या मागची 50 पानी index पुस्तक जाड करते). (2) **लिहिण्याचा वेग:** आता प्रत्येक नव्या ऑर्डरला **दोन** जागी लिहावं लागतं: मूळ नोंद आणि अनुक्रमणिका (नवा शब्द आला की index पुस्तकातही भर घालावी लागते). म्हणजे index वाचन जलद करते पण लेखन हळू. (3) **सांभाळ:** प्रत्येक index राखावा लागतो. निष्कर्ष: ****index = वाचन-वेग विकत, लेखन-वेग**

- जागा विकून.**

आणि इथे नवख्याची खरी परीक्षा: “मग सगळ्या स्तंभांवर index टाकू, सगळं जलद!” चूक. वाचनाला जलद, पण **प्रत्येक** लेखन तेवढ्याच index मध्ये भर घालत बसतं: 15 index असलेली table मध्ये एक ऑर्डर लिहायला 15 अनुक्रमणिका update: लेखन कासवगती. Top engineer index टाकतो **जिथे वारंवार शोध होतो तिथेच:** रमेशचा नंबर, ऑर्डरची तारीख: हो; कचित शोधले जाणारे स्तंभ: नको. **कुठे index, कुठे नाही:** हा नजरेचा निर्णय आहे, आणि तो सौदा बघण्यातून येतो.

महाग चूक

”App हळू आहे = मोठा server घेऊ (जास्त पैसे).” आरतीने जवळपास हीच चूक केली: हळूपणा बघून महागडा server भाड्याने घ्यायला निघाली. पण खरी अडचण server ची ताकद नव्हती, **गहाळ index** होती: एका ओळीच्या index ने 10x महागड्या server पेक्षा जास्त वेग दिला, फुकट. **हळूपणाचं पहिलं निदान “मोठा server” नसून “कुठली शोध-रचना” हे असतं.** पैसे ओतण्याआधी नजर टाका: software मधल्या 90% हळूपणाचं मूळ रचनेत असतं, ताकदीत नाही.

नकाशावर

- 1.1 एक यंत्र, तीन कामं; एकच अपयश-बिंदू
- 1.2 database = शिस्तीची वही (ACID); खात्री विकत, वेग विकून
- 1.3 index = अनुक्रमणिका; वाचन जलद, लेखन+जागा विकून;
जिथे शोध तिथेच

सूत्रावर: index हा वाचन-वि.-लेखन चा शुद्ध सौदा: तुमचं app जास्त काय करतं (वाचतं की लिहितं) यावर निर्णय. सौदा न बघता “सगळीकडे index” म्हणणारा नवखा; “इथे वाचन जास्त म्हणून index” म्हणणारा जाणकार.

स्वतः बघा (5 मिनिटं)

कुठलंही पाठ्यपुस्तक उघडा आणि मागची अनुक्रमणिका/index बघा. एक शब्द निवडा, index वापरून शोधा (2 सेकंद), मग तोच शब्द पहिल्या पानापासून चाळत शोधायचा कल्पना करा (मिनिटं). हाच फरक database मध्ये milliseconds वि. सेकंद. आणि लक्षात घ्या: त्या index पानांनी पुस्तक जाड केलंय: तीच जागेची किंमत.

विचार करा

आरतीचं app आणखी वाढलं: दिवाळी आली, एका दिवसात 50,000 ऑर्डर. Index असूनही database धापा टाकू लागली: कारण हजारो लोक **एकाच वेळी** तीच लोकप्रिय माहिती (आजचे भाव, सणाच्या ऑफर) पुन्हा-पुन्हा विचारत होते. Database प्रत्येक वेळी तीच गोष्ट नव्याने शोधून देत होती. आरती विचारते: “एकच उत्तर हजार वेळा तेच असेल, तर दर वेळी नव्याने का शोधायचं?”

बरोबर पकडलंस. तेच-तेच उत्तर हातासरशी ठेवायची युक्ती म्हणजे cache: आणि ती software मधली सगळ्यात शक्तिशाली, आणि सगळ्यात **धोकादायक** युक्ती आहे. पुढची भिंत.

Chapter 1.4 [SPINE]: Cache: हातासरशी उत्तर, आणि त्याचं दुखणं

Cache ही software मधली सगळ्यात मोठी वेग-देणारी युक्ती आहे, आणि तिच्याबद्दल एक जगप्रसिद्ध विनोद आहे: ”**Computer science मध्ये फक्त दोनच अवघड गोष्टी आहेत: cache मधून गोष्ट कधी काढायची, आणि गोष्टींना नावं देणं.**” का अवघड, ते आरतीच्या दिवाळीत दिसेल.

Cache म्हणजे काय. दुकानदाराची उपमा: रोज विचारला जाणारा भाव (साखर, तेल) दुकानदार गोदामात जाऊन बघत नाही; तो गल्ल्याजवळ एका कागदावर लिहून ठेवतो: **हातासरशी**. Cache तेच: वारंवार लागणारं उत्तर database मध्ये न जाता एका **जलद, जवळच्या** जागी (memory त) ठेवायचं. आरती आजचे भाव, लोकप्रिय दुकानांच्या याद्या cache मध्ये ठेवते: हजारो लोक विचारले तरी database ला त्रास नाही, उत्तर गल्ल्याच्या कागदावरून. दिवाळीचा भार पेलला. Cache मुळे वाचनं 10-100 पट जलद-स्वस्त होऊ शकतात: म्हणून प्रत्येक मोठं app cache वापरतं.

पण इथे सूत्राचं सगळ्यात कपटी रूप: cache = वेग विकत, ताजेपणा विकून. गल्ल्यावरचा कागद जुना झाला तर? साखरेचा भाव गोदामात बदलला पण कागदावर जुनाच राहिला: दुकानदार चुकीचा भाव सांगतो. Cache मधली माहिती आणि खरी (database मधली) माहिती **वेगळी पडणं**: याला **stale cache** (शिळं cache) म्हणतात, आणि हेच ते “कधी काढायची” चं दुखणं. आरतीच्या app मध्ये हे घडलं: एका दुकानाने साखर संपली म्हणून काढून टाकली, पण cache मध्ये “उपलब्ध” राहिली: रमेशने ऑर्डर केली, दुकानात गेल्यावर “नाही” म्हणाले. एक शिळ्या-cache ची चूक = एक नाराज गिन्हाईक.

म्हणून cache चा खरा प्रश्न: जुनं कधी फेकायचं (invalidation). तीन ढोबळ रीती (प्रत्येकीला सौदा): (1) **वेळ ठरवा (TTL):** “हा भाव 5 मिनिटं ताजा माना, मग नव्याने आणा”: सोपं, पण 5 मिनिटं जुनी माहिती चालेल तरच. (2) **बदललं की फेका:** भाव database मध्ये बदलला की cache मधून लगेच काढा: ताजं, पण गुंतागुंतीचं (प्रत्येक बदलाला cache-ला कळवावं लागतं). (3) **जागा संपली की जुनं-न-वापरलेलं फेका:** cache भरली की सगळ्यात कमी वापरलेलं बाहेर. निवड कशावर? ”**किती जुनी माहिती चालेल**” यावर: भावाला काही मिनिटं चालतं; शिल्लक/पैसे यांना **शून्य** शिळेपणा (तिथे cache टाळाच, किंवा अति-कडक).

नवख्याचा धडा (आरतीने घेतला): cache हे “app हळू झालं की टाकायचं मलम” नाही. **आधी विचारा: ही माहिती किती वेळ जुनी चालेल?** “अजिबात नाही” (पैसे) तर cache नको; “काही मिनिटं चालेल” (भाव, याद्या) तर cache सोनं. हा एक प्रश्न न विचारता cache टाकणारे engineers शिळ्या-माहितीचे bugs वर्षानुवर्ष शोधत बसतात: कारण bug **कधीतरीच** दिसतो (cache ताजं असताना नाही, शिळं असताना), म्हणून पकडणं कठीण.

महाग चूक

"Cache टाकलं, app जलद झालं, काम फत्ते." Cache टाकणं सोपं; **कधी काढायचं** ठरवणं अवघड: आणि तेच न ठरवता cache टाकणारे सगळ्यात कपटी bug विकत घेतात: गिन्हाईकाला चुकीचा भाव/शिल्लक, जो तुमच्या testing मध्ये कधीच दिसत नाही (कारण तुमचं cache ताजं असतं). Stale cache चे bugs हे software मधले सर्वात जास्त वेळ लपून राहणारे bugs आहेत. Cache टाकताना पहिला प्रश्न "जलद होईल का" नाही, **"जुनं कधी, कसं फेकायचं"** हा आहे.

नकाशावर

- 1.1 एक यंत्र, तीन कामं 1.2 ACID: खात्री विकत
- 1.3 index: वाचन जलद, लेखन विकून
- 1.4 cache: वेग विकत, ताजेपणा विकून; खरा प्रश्न "जुनं कधी फेकायचं"; "किती जुनं चालेल?" आधी विचारा

सूत्रावर: cache हा सगळ्यात मोहक सौदा (प्रचंड वेग) आणि सगळ्यात लपलेली किंमत (शिळेपणा). जो "किती जुनं चालेल" आधी विचारतो तो cache चा मालक; जो नुसतं वेगासाठी टाकतो तो cache चा गुलाम.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनवर कुठलंही app उघडा, मग "refresh"/खाली-ओढून- ताजं-करा. जे नवं दिसतं ते cache मध्ये जुनं होतं: तुम्ही ते आत्ता फेकलंत (manual invalidation!). कुठली माहिती refresh शिवाय जुनी राहते (cache), कुठली नेहमी ताजी येते (no cache): याचा अंदाज घ्या. App-designers नी कुठे "किती जुनं चालेल" ठरवलंय ते तुम्हाला दिसेल.

विचार करा

आरतीचं app आता चांगलं चाललंय. पण एका दुपारी तिचा एकुलता एक server (cloud मधला) बंद पडला: hardware बिघाड. **पूर्ण app बंद**, दोन तास. गिन्हाईकं, दुकानं, सगळे अडकले. आरती हादरली: "एकाच यंत्रावर पूर्ण धंदा: हे धोक्याचं आहे. पण एकच database आहे: तिच्या दोन प्रती कशा ठेवणार? दोन्ही एकसारख्या कशा राहणार?"

आरती आत्ता 1.1 चा “एकच अपयश-बिंदू” प्रत्यक्ष भोगतेय. उत्तर: database ची प्रत ठेवायची: याला replication म्हणतात. पण “दोन्ही एकसारख्या कशा राहणार” हा तिचा प्रश्न या पुस्तकातला सगळ्यात खोल प्रश्नांपैकी एक आहे: पुढची भिंत.

Chapter 1.5 [SPINE]: Replication: प्रती आणि त्यांचा उशीर

आरतीच्या दोन-तास-बंद च्या धक्क्यातून पुढची भिंत: एकच यंत्र म्हणजे एकच अपयश-बिंदू (1.1 चा इशारा खरा ठरला). उपाय सोपा वाटतो: **प्रत ठेवा**. पण त्या “सोप्या” उपायात software मधलं एक खोल सत्य दडलंय.

Replication म्हणजे काय. दुकानाच्या वहीची उपमा: महत्त्वाची वही एकच असेल आणि जळली तर सर्वनाश. म्हणून शहाणा दुकानदार रोज एक **कार्बन-प्रत** दुसऱ्या दुकानात ठेवतो. Database replication तेच: मुख्य database (primary) च्या नोंदी लगोलग एक-दोन प्रतींमध्ये (replicas) उतरतात. फायदे दोन: (1) मुख्य बंद पडली तर प्रत ताबा घेते (आरतीची दोन-तास-बंद टळते); (2) **वाचन वाटता येतात**: हजारो “भाव बघा” च्या विनंत्या प्रतींकडे वळवा, मुख्य फक्त लेखनासाठी: भार हलका.

पण इथे तो खोल प्रश्न: प्रत किती मागे असते? कार्बन-प्रत दुसऱ्या दुकानात पोहोचायला वेळ लागतो: तेवढा वेळ ती प्रत **जुनी** असते. Database मध्ये हा उशीर milliseconds चा असतो, पण **शून्य नसतो**: याला **replication lag** म्हणतात. आणि इथून एक विचित्र, खरा प्रसंग जन्मतो: रमेशने ऑर्डर दिली (मुख्य database वर लिहिली गेली); लगेच “माझ्या ऑर्डर” बघितलं (ती विनंती एका **प्रती**कडे गेली, जी अजून 200 milliseconds मागे होती): आणि रमेशला **स्वतःचीच ऑर्डर दिसली नाही!** “आत्ताच तर केली, गेली कुठे?” त्याने घाबरून पुन्हा ऑर्डर केली: **दुहेरी ऑर्डर**. एका छोट्या उशिराने खरा गोंधळ.

सूत्र इथे नव्या रूपात: प्रती = टिकाव + वाचन-वेग विकत, “ताजेपणाची खात्री” विकून. आणि यातून एक निवड जन्मतो जी पुढच्या दोन chapters ना (CAP, consistency) हात घालते: **रमेशला त्याची स्वतःची ताजी ऑर्डर दाखवणं महत्त्वाचं** (मग त्याची वाचनं मुख्य database कडेच पाठवा, प्रतीकडे नको: “स्वतःचं लिखाण स्वतःला ताजं दिसो” हा नियम); पण **”शेजारच्या गावातल्या दुकानाची यादी 200ms जुनी दिसली तर चालेल** (ती प्रतीकडून घ्या, स्वस्त). एकाच app मध्ये दोन वेगळे निर्णय, माहितीच्या स्वभावानुसार. हेच रचनेचं सौंदर्य: **एकच नियम सगळीकडे लावणं म्हणजे नवखेपणा.**

खरं जग: वाचन-भार वाटणं इतकं मोठं का. बहुतेक app मध्ये वाचनं लेखनांपेक्षा **कितीतरी पट** जास्त असतात (एक ऑर्डर, पण ती-आधी 50 वेळा भाव-याद्या बघितल्या). म्हणून replicas कडे वाचनं वळवणं ही सगळ्यात स्वस्त, सगळ्यात परिणामकारक पहिली scaling-चाल आहे: मुख्य database चा भार नाटकीय घटतो. WhatsApp, बँका, प्रत्येक मोठं app हे करतं. पण lag चं भान ठेवून: **कुठलं वाचन ताजं हवं, कुठलं जुनं चालेल.**

महाग चूक

”प्रती ठेवल्या, आता सगळी वाचनं प्रतींकडे, मुख्य फक्त लेखनाला: जास्तीत जास्त भार-वाटप!” उत्साहात हे केलं की रमेशची-दुहेरी- ऑर्डर सारखे bugs पेरले जातात: वापरकर्त्याला **स्वतःचं ताजं लिखाण दिसत नाही** (read-after-write चा भंग), आणि तो गोंधळतो, पुन्हा करतो, तक्रार करतो. नियम: **वाचन प्रतीकडे वळवताना “हे वाचन किती जुनं चालेल” विचारा**: वापरकर्त्याचं स्वतःचं ताजं काम मुख्याकडून; बाकीचं जग प्रतीकडून. सरसकट भार-वाटप म्हणजे सरसकट गोंधळ.

नकाशावर

- | | |
|---|--------------------------|
| 1.1 एक यंत्र, एकच अपयश-बिंदू | 1.2 ACID |
| 1.3 index: वाचन जलद | 1.4 cache: ताजेपणा विकून |
| 1.5 replication: प्रती = टिकाव + वाचन-वेग विकत, ताजेपणाची खात्री विकून (lag); ताजं-हवं मुख्याकडे, जुनं-चालेल प्रतीकडे | |

सूत्रावर: replication ने आरतीने टिकाव विकत घेतला, पण “क्षणाची अचूकता” विकली. आणि तिला कळलं की माहितीच्या स्वभावानुसार वेगवेगळे सौदे करावे लागतात: हीच प्रौढ नजर.

स्वतः बघा (5 मिनिटं)

WhatsApp वर एखाद्या group मध्ये message पाठवा आणि लक्ष द्या: तुम्हाला तो **लगेच** दिसतो (तुमचं ताजं लिखाण), पण समोरच्याला पोहोचायला (आणि दोन निव्व्या खुणा यायला) क्षणभर जास्त. ते अंतर म्हणजे तुमच्या-फोनवरचं-ताजेपण आणि जगात- पसरण्याचा-उशीर यातलं अंतर: replication lag चा रोजचा अनुभव.

विचार करा

आरतीचं app आता तीन शहरांत, 5 लाख ऑर्डर रोज. प्रतींनी वाचन-भार पेलला, पण आता **लेखनच** इतकी झाली की एकच मुख्य database धापा टाकतेय: सगळ्या ऑर्डर शेवटी तिच्याकडेच लिहाव्या लागतात (प्रती फक्त वाचनाला). आरती विचारते: “वाचनं वाटता आली, पण लेखनं एकाच जागी: ती कशी वाटणार? एकाच वहीत सगळं जग लिहितंय.”

हाच scaling चा सगळ्यात अवघड टप्पा: लेखन वाटणं. उत्तर: एकच प्रचंड वही न ठेवता तिचे **तुकडे** करायचे: शहरागणिक, पत्त्यागणिक. याला sharding म्हणतात, आणि त्यात एक कपटी सापळाही आहे. पुढची भिंत.

Chapter 1.6 [SPINE]: Sharding: वहीचे तुकडे आणि Hotspot चा सापळा

आरतीच्या “एकाच वहीत सगळं जग लिहितंय” या पेचावर उत्तर: वहीचे तुकडे करा. सोपं वाटतं; पण इथे software मधला एक सुंदर आणि कपटी धडा आहे.

Sharding म्हणजे काय. मोठ्या शहराच्या एकाच पोस्ट-ऑफिसची उपमा: सगळी पत्रं एकाच ठिकाणी आली तर रांग अनंत. उपाय: **विभागवार पोस्ट-ऑफिस**: कोथरूडची पत्रं कोथरूड-ऑफिसला, कॅप ची कॅप-ला. प्रत्येक ऑफिस फक्त स्वतःचा भाग सांभाळतं; भार वाटला. Database sharding तेच: एक प्रचंड table अनेक यंत्रांवर **तुकड्यांत** वाटायची. आरती शहरागणिक तुकडे करते: पुण्याच्या ऑर्डर एका यंत्रावर, मुंबईच्या दुसऱ्यावर, नाशिकच्या तिसऱ्यावर. आता लेखनही वाटली गेली: प्रत्येक यंत्र फक्त स्वतःच्या शहराचं ओझं उचलतं. Scaling ची सर्वात मोठी भिंत ओलांडली.

तुकडा कशाने पाडायचा: हा निर्णय सगळं ठरवतो (shard key). आरतीने “शहर” निवडलं. पण समजा तिने “ऑर्डरची तारीख” निवडली असती: मग **आजच्या** सगळ्या ऑर्डर एकाच यंत्रावर आदळल्या असत्या (सगळे आजच लिहितात!), बाकी यंत्रं रिकामी. एका यंत्रावर गर्दी, बाकी बसून: याला **hotspot** (तापलेला तुकडा) म्हणतात, आणि हा sharding चा सगळ्यात सामान्य सापळा आहे. **चांगला shard key तो जो भार सगळ्या तुकड्यांत सारखा वाटतो.** शहर बरं (सगळी शहरं एकसारखी वापरली तर); पण जर 80% धंदा पुण्यातच असेल, तर “शहर” सुद्धा पुण्याचा तुकडा तापवेल. **Shard key ची निवड = भार कसा पसरेल याचं भाकीत:** चुकला की एकच यंत्र जळतं आणि “sharding केलं तरी हळू” हे कोडं पडतं.

आणि sharding ची जबर किंमत (सूत्र सर्वात महाग इथे): तुकडे केले की एक गोष्ट **फार** अवघड होते: **दोन तुकड्यांना मिळून प्रश्न विचारणं.** “सगळ्या शहरांतली आजची एकूण विक्री किती?": आता हे एका वहीत नाही; प्रत्येक यंत्राला विचारून बेरीज करावी लागते: हळू, गुंतागुंतीचं. आणि “एका shard वरून दुसऱ्याकडे पैसे हलवा” (ACID व्यवहार दोन यंत्रांत!): हे तर भयंकर अवघड. म्हणून धडा: **sharding = लेखन-वाढ विकत, “सोपे एकत्रित प्रश्न” विकून.** म्हणूनच हा शेवटचा उपाय आहे, पहिला नाही: आधी index, cache, replicas (स्वस्त, सोपे); sharding तेव्हाच जेव्हा लेखनच यंत्राच्या ताकदीबाहेर जातात.

खरं जग: का हा शेवटचा उपाय. आरतीने योग्य क्रम पाळला: 5 लाख ऑर्डरपर्यंत index+cache+replicas पुरले; sharding तिने **50 लाख** ऑर्डरला, नाईलाजाने केलं: कारण एकदा sharded केलं की app चं code कायमचं गुंतागुंतीचं होतं (प्रत्येक प्रश्नाला “कुठला तुकडा” विचार). जगातल्या अनुभवी teams चा नियम: **”जोवर टाळता येईल तोवर sharding टाळा.”** नवखे उत्साहात आधीच shard करतात आणि विनाकारण गुंतागुंत ओढवून घेतात.

महाग चूक

”मोठं app म्हणजे sharding, आधीपासूनच करू.” Premature sharding (वेळेआधी तुकडे): 1.1 च्या premature scaling चा जुळा भाऊ. दहा-लाख ऑर्डर नसताना sharding म्हणजे: सोपे प्रश्न कायमचे अवघड, कोड कायमचं गुंतागुंतीचं, आणि तरी फायदा शून्य (कारण एक यंत्र पुरतंच होतं). Sharding चा खरा संकेत एकच: **लेखनंच** एका (शक्तिशाली) यंत्राच्या आवाक्याबाहेर. तोवर index-cache-replicas: स्वस्त, सोपे, पुरेसे.

नकाशावर

- 1.1-1.4 यंत्र | ACID | index | cache
- 1.5 replication: वाचन वाटलं
- 1.6 sharding: लेखन वाटलं (shard key = भार-वाटपाचं भाकीत; hotspot चा सापळा); लेखन-वाढ विकत, एकत्रित प्रश्न विकून; शेवटचा उपाय

सूत्रावर: sharding हा सर्वात मोठा सौदा: प्रचंड लेखन-क्षमता मिळवायला “साधेपणा” कायमचा विकावा लागतो. म्हणून तो जपून, उशिरा, नाईलाजाने: सौदा किती मोठा हे बघून निर्णय.

स्वतः बघा (5 मिनिटं)

तुमच्या शहरातल्या मोठ्या सेवेचा विचार करा (गॅस-कनेक्शन, वीज- बिल): त्यांचे विभागवार office असतात ना? तुमचं काम तुमच्याच विभागात होतं (जलद), पण “पूर्ण शहराची आकडेवारी” काढायला सगळ्या office ना विचारावं लागतं (हळू). हेच sharding: विभागणीने रोजचं काम जलद, पण एकत्रित चित्र अवघड. तुम्ही हे रोज अनुभवता.

विचार करा

आरतीचं app आता प्रचंड आहे. पण एका संध्याकाळी दिवाळीला लाखो ऑर्डर एकदम आल्या आणि app कोसळलं: server, database सगळं तयार होतं, पण ऑर्डर इतक्या **वेगात** आल्या की process करायला वेळच मिळेना: एकावर एक रचल्या गेल्या, गोंधळ. आरती विचारते: “गिऱ्हाईकाची ऑर्डर लगेच ‘घेतली’ म्हणायला हवी, पण ती पूर्ण करायला (दुकानाला कळवणं, delivery ठरवणं) वेळ लागतो. हे दोन्ही एकाच वेळी, एकाच वेगात कसं जमणार?”

जमणार नाही, आणि जमवायचंही नाही: ती दोन कामं **वेगळी** करायची. “ऑर्डर घेतली” लगेच सांगा; “ऑर्डर पूर्ण करा” एका रांगेत टाका, आरामात उरका. या रांगेचं नाव queue, आणि ती मोठ्या systems ची कणा आहे. पुढची भिंत.

Chapter 1.7 [SPINE]: Queue: रांगा, आणि “पुन्हा-पुन्हा चालणारं” काम

आरतीच्या दिवाळी-कोसळण्यातून एक संकल्पना जन्मते जी मोठ्या systems चा कणा आहे: **queue** (रांगा). आणि तिच्यासोबत एक जुळी संकल्पना, जिच्याशिवाय queue थोकादायक: **idempotency**.

दोन कामं वेगळी करा. आरतीचा पेच: “ऑर्डर घेतली” हे लगेच सांगायचंय (गिन्हाईक वाट बघतंय), पण “ऑर्डर पूर्ण करणं” (दुकानाला sms, delivery-boy ठरवणं, नकाशा काढणं) याला वेळ लागतो. हॉटेलची उपमा: वेटर तुमची ऑर्डर घेतो आणि **लगेच** “आलं साहेब” म्हणतो (तुम्ही निश्चित); मग ती चिठ्ठी स्वयंपाकघराच्या **खुंटीवर** लावतो; आचारी सवडीने, क्रमाने बनवतो. वेटर प्रत्येक ऑर्डर बनेपर्यंत उभा राहिला तर हॉटेल ठप्प. ती **खुंटी** म्हणजे queue: कामं तिथे रांगेत बसतात, आणि मागचे workers ती एकेक उचलून करतात, स्वतःच्या वेगाने.

Queue काय विकत घेते. (1) **धक्का शोषून घेणं:** दिवाळीला लाखो ऑर्डर एकदम आल्या तरी त्या रांगेत शांत बसतात; workers जमेल तसं उरकतात: app कोसळत नाही, फक्त थोडा उशीर. पुराचं पाणी धरणात साठवण्यासारखं. (2) **कामं तोडणं:** ऑर्डर-घेणं आणि ऑर्डर-पूर्ण-करणं वेगळे झाले; एक हळू झालं तरी दुसरं थांबत नाही. (3) **अपयश सोसणं:** delivery-यंत्रणा 2 मिनिटं बंद पडली तरी ऑर्डर रांगेत सुरक्षित; परत आल्यावर उचलल्या जातात: काहीच हरवत नाही.

पण इथे कपटी किंमत: काम दोनदा होऊ शकतं. Queue मध्ये एक विचित्र गोष्ट घडते: worker ने ऑर्डर उचलली, काम केलं, पण “झालं” कळवण्याआधीच तो कोसळला: queue ला वाटतं काम झालंच नाही, ती **तीच ऑर्डर दुसऱ्या worker ला** देते. आता रमेशला **दोन** पोती साखर, दोनदा पैसे. Queue ची हमी बहुधा “किमान एकदा” (at-least-once) असते: म्हणजे “एकदा तरी होईल, कदाचित दोनदाही.” भयंकर वाटतं, पण उपाय सुंदर आहे:

Idempotency: दोनदा केलं तरी परिणाम एकदाच. लिफ्टचं बटण पाच वेळा दाबा; लिफ्ट एकदाच येते: बटण **idempotent** आहे. आरती प्रत्येक ऑर्डरला एक **एकमेव क्रमांक** देते; worker काम करण्याआधी बघतो “हा क्रमांक आधी केलाय का?": केला असेल तर सोडून देतो. आता तीच ऑर्डर दोनदा आली तरी साखर एकदाच, पैसे एकदाच. **मोठ्या systems मध्ये “काम दोनदा येऊ शकतं” हे गृहीत धरून प्रत्येक कृती idempotent बांधणं:** ही top engineer ची सवय आहे; नवखा “दोनदा येणारच नाही” असं गृहीत धरतो आणि दुहेरी-payment चे bugs पेरतो.

महाग चूक

”Queue टाकली म्हणजे विश्वासार्ह झालं, निश्चित.” Queue विश्वासार्हता देते पण **”काम एकदाच होईल” ची हमी देत नाही**: उलट “दोनदा होऊ शकतं” हे तिचं डिफॉल्ट वर्तन आहे. हे न समजता queue वापरणारे सर्वात वाईट bugs पेरतात: दुहेरी पैसे, दुहेरी sms, दुहेरी ऑर्डर: आणि हे bugs फक्त गर्दीच्या वेळी (worker कोसळण्याच्या क्षणी) दिसतात, म्हणून testing मध्ये कधीच सापडत नाहीत. Queue वापरायची तर idempotency **सोबतच**: एक नाण्याच्या दोन बाजू.

नकाशावर

- 1.1-1.4 यंत्र | ACID | index | cache
- 1.5-1.6 replication (वाचन) | sharding (लेखन)
- 1.7 queue: धक्का शोषतो, कामं तोडतो; पण “किमान एकदा” =
दोनदा शक्य -> idempotency (दोनदा केलं तरी परिणाम एकदाच)

सूत्रावर: queue ने आरतीने लवचिकता-टिकाव विकत घेतले, पण “लगेच झालं” ची खात्री विकली (आता “होईल, थोड्या वेळात”). आणि “किमान एकदा” ही किंमत idempotency ने भरून काढली: प्रत्येक सोयीमागे सौदा, प्रत्येक सौद्यावर उपाय.

स्वतः बघा (5 मिनिटं)

तुम्ही ऑनलाइन payment करताना “back” दाबलंत किंवा page refresh केलंत तर “दोनदा पैसे कापले जातील का” ही भीती वाटते ना? चांगल्या app मध्ये कापले जात नाहीत: कारण ते idempotent आहेत (तोच व्यवहार-क्रमांक ओळखून दुसऱ्यांदा टाळतात). पुढच्या वेळी एखादं app “तुमची विनंती processing आहे, थांबा” म्हणतं तेव्हा ओळखा: मागे queue चालू आहे.

विचार करा

आरतीचं app आता प्रचंड. पण एका गोष्टीचा ताण वाढला: लाखो गिन्हाईकांच्या विनंत्या **कुठल्या** server कडे पाठवायच्या? आता एक server नाही, वीस आहेत (भार पेलायला). पण गिन्हाईकांच्या फोनला कसं कळणार कुठला server मोकळा आहे? आणि एखादा server बंद पडला तर विनंत्या तिकडे जाऊच नयेत, हे कोण बघणार?

एक “वाहतूक-पोलिस” लागतो: जो येणाऱ्या विनंत्या मोकळ्या servers कडे वळवतो, बंद पडलेले टाळतो. त्याचं नाव load balancer, आणि तो प्रत्येक मोठ्या app च्या दारात उभा असतो. पुढची भिंत.

Chapter 1.8 [SPINE]: Load Balancer: वाहतूक-पोलिस

आरतीच्या “कुठल्या server कडे पाठवायचं” या पेचावर उत्तर एका साध्या पण अपरिहार्य यंत्रात आहे: **load balancer**. आणि त्याच्यासोबत एक कल्पना जी मोठ्या systems चा पाया आहे: **horizontal scaling**.

दोन मार्गांनी मोठं होणं. आरतीचा एक server कमी पडतोय. दोन उपाय: (1) **तोच server अधिक शक्तिशाली करा** (जास्त RAM, वेगवान CPU): याला vertical scaling म्हणतात: एक बैल अधिक ताकदीचा करणं. सोपं, पण मर्यादा आहे (जगातला सर्वात शक्तिशाली एकच बैल किती ओढेल?) आणि महाग, आणि तो एकच बैल पडला तर पुन्हा सर्वनाश. (2) **अनेक साधे servers लावा** आणि भार वाटा: horizontal scaling: एका महाबलाऐवजी वीस सामान्य बैल. हेच मोठ्या systems चं रहस्य: **अनेक स्वस्त यंत्रं > एक महागडं यंत्र**: कारण अमर्याद वाढता येतं (अजून बैल जुंपा) आणि एक पडला तरी बाकी ओढतात.

पण अनेक बैल जुंपले की एक अडचण: येणारी विनंती कुठल्या बैलाकडे? इथे load balancer येतो: app च्या दारात उभा वाहतूक-पोलिस. प्रत्येक विनंती तो बघतो आणि **मोकळ्या** server कडे वळवतो: कधी आळीपाळीने (एकेकाला एक), कधी “सर्वात कमी भार असलेल्याकडे.” आणि सर्वात महत्वाचं: तो सतत **तब्येत तपासतो** (health check): एखादा server बंद पडला की त्याला विनंत्या पाठवणं **थांबवतो**: गिन्हाईकाला कळतही नाही की एक server मेला. आरतीची 1.4 मधली दोन-तास-बंद आता अशक्य: एक server पडला तरी वाहतूक-पोलिस बाकी एकोणीसकडे वळवतो.

यासाठी एक अट: servers “विसराळू” हवेत (stateless). वाहतूक-पोलिस कुठल्याही server कडे विनंती पाठवू शकतो, तेव्हाच जेव्हा **सगळे servers सारखे** असतात: कुणाकडेही खास माहिती साठलेली नसते. जर रमेशची “cart” (निवडलेल्या वस्तू) एकाच server च्या memory त साठली, तर पुढची विनंती दुसऱ्या server कडे गेली की cart गायब! म्हणून नियम: **servers ने काहीही स्वतःकडे साठवायचं नाही; सगळी माहिती database/cache मध्ये (सामायिक जागी).** मग प्रत्येक server “विसराळू पण सारखा”: कुठलाही कुठलीही विनंती हाताळू शकतो. हे stateless तत्त्व horizontal scaling चा पाया आहे: याशिवाय अनेक-servers चं गणित कोसळतं.

सूत्र इथे: horizontal scaling = अमर्याद वाढ + टिकाव विकत, पण “साधेपणा” विकून (आता अनेक यंत्रं, load balancer, stateless- शिस्त सांभाळा). लहान app ला एक server सोपा; मोठ्याला अनेक-पण-गुंतागुंतीचे. पुन्हा तोच सूत्र: सोपेपणा आणि वाढ यांचा सौदा, आता उलट दिशेने.

महाग चूक

”Server ची memory आहेच, त्यात गिन्हाईकाची माहिती (login, cart) ठेवू: जलद मिळेल.” हे stateful servers: horizontal scaling चा सगळ्यात सामान्य घात. एका server वर माहिती साठवली की load balancer ला त्याच गिन्हाईकाला **त्याच** server कडे पाठवावं लागतं (sticky): मग तो server पडला की त्या सगळ्या गिन्हाईकांची माहिती गेली, आणि भार-वाटपही बिघडलं. नियम: **server विसराळू ठेवा; माहिती सामायिक जागी**. ही एक शिस्त पाळली की तुमचं app वीस काय, दोनशे serversपर्यंत सहज वाढतं.

नकाशावर

1.1-1.4 यंत्र | ACID | index | cache
 1.5-1.7 replication | sharding | queue
 1.8 load balancer = वाहतूक-पोलिस (मोकळ्याकडे वळवतो, मेलेले टाळतो); horizontal scaling (अनेक स्वस्त > एक महाग);
 अट: servers विसराळू (stateless)

सूत्रावर: load balancer + stateless servers नी आरतीने “एकच अपयश-बिंदू” कायमचा संपवला (1.1 चं वर्तुळ पूर्ण): पण साधेपणा विकून. आता तिचं app अनेक यंत्रांचं जिवंत जाळं आहे.

स्वतः बघा (5 मिनिटं)

मोठ्या दुकानात/बँकेत गेलात तर “पुढील ग्राहक: counter 3” अशी व्यवस्था बघितली असेल: तो load balancer आहे! एकच रांग, अनेक counters, आणि व्यवस्थापक तुम्हाला मोकळ्या counter कडे पाठवतो; एखादा counter बंद झाला तर तिथे कुणाला पाठवत नाही. Software मध्ये अगदी हेच, फक्त milliseconds मध्ये आणि आपोआप.

विचार करा

आरतीचं app आता भारतभर. पण दिल्लीच्या गिन्हाईकाला app उघडायला मुंबईच्या पेक्षा जास्त वेळ लागतोय: कारण आरतीचे सगळे servers मुंबईत, आणि दिल्लीहून विनंती मुंबईला जाऊन-येऊन अंतर कापते (Book 1: latency = अंतर). आरती विचारते: “माहिती तीच आहे, मग दिल्लीवाल्याला हळू का? आणि दुकानाचे फोटो, app ची images: ती तर प्रत्येक वेळी मुंबईहून का पाठवायची?”

कारण अंतर हेच latency, आणि प्रकाशाच्या वेगालाही मर्यादा आहे. उपाय: गिन्हाईकाच्या **जवळ** माहितीची प्रत ठेवायची: विशेषतः न-बदलणारी (फोटो, images). याचं नाव CDN, आणि ते internet ला वेगवान ठेवणारं अदृश्य जाळं आहे. विभागाची शेवटची भिंत.

Chapter 1.9 [SPINE]: CDN: अंतर कापणारं जाळं

आरतीच्या “दिल्लीवाल्याला हळू का” या पेचावर उत्तर एका साध्या सत्यात आहे: **अंतर हेच वेळ**. आणि ते कापणारं जाळं म्हणजे CDN.

अंतराचं गणित (Book 1 खोल). प्रकाश जगातला सर्वात वेगवान, पण त्यालाही मर्यादा: मुंबई-दिल्ली सिग्नल जाऊन-यायला काही milliseconds; मुंबई-अमेरिका ~200-250 milliseconds (तपासलेला आकडा). एका विनंतीत हे ठीक; पण एक page उघडायला अनेक विनंत्या (images, फोटो, याद्या): प्रत्येकीला अंतराचा कर: दिल्लीवाल्याचं app रेंगाळतं. आणि हा उशीर server च्या ताकदीचा नाही, **भूगोलाचा** आहे: जगातला वेगवान server सुद्धा दूर असेल तर हळूच.

CDN म्हणजे काय. Content Delivery Network: गिन्हाईकाच्या **जवळ** माहितीच्या प्रती ठेवणारं, जगभर पसरलेलं यंत्र-जाळं. पाणपोईची उपमा: प्रत्येक तहानलेल्याला विहिरीपर्यंत (मुंबई-server) पाठवण्याऐवजी, गावोगाव पाणपोई ठेवायची: तहान जवळच भागते. आरती दुकानांचे फोटो, app च्या images, न-बदलणारं सगळं CDN वर टाकते: दिल्लीवाल्याला ते **दिल्लीच्या** पाणपोईतून मिळतं, मुंबईपर्यंत जावं लागत नाही. app भुरकन उघडतं.

महत्त्वाचा भेद: काय CDN वर जातं, काय नाही. CDN चांगलं काम करतं **न-बदलणाऱ्या** (static) गोष्टींसाठी: फोटो, video, app चं design, images: हे सगळ्यांना सारखं आणि क्वचित बदलतं, म्हणून जवळ प्रत ठेवणं सुरक्षित. पण **बदलणारी, वैयक्तिक** माहिती (रमेशची आजची ऑर्डर, त्याची शिल्लक) CDN वर ठेवता येत नाही: ती प्रत्येकाची वेगळी आणि सतत बदलणारी (1.4 चं cache-invalidation चं दुखणं परत!). म्हणून नियम: **static जवळ (CDN), dynamic मुळाशी (server)**. आरतीचं app अर्थ जवळून (design, फोटो: जलद), अर्थ मुळापासून (तिची ऑर्डर: ताजी) येतं: दोन्हीचं उत्तम.

सूत्र इथे: CDN = वेग (जवळीक) विकत, ताजेपणा + जागा विकून. हे cache चं भौगोलिक रूप आहे (1.4): जगभर प्रती, म्हणून जगभर तोच “जुनं कधी फेकायचं” चा प्रश्न: फोटो बदलला तर सगळ्या पाणपोईंना कळवायला वेळ लागतो. पण static गोष्टी क्वचित बदलतात, म्हणून हा सौदा बहुधा सहज फायद्याचा.

विभाग-अखेरचं मोठं चित्र: आरती इथवर पोहोचली: एक laptop पासून जगभर पसरलेल्या जाळ्यापर्यंत. आणि प्रत्येक भिंतीने एक सौदा शिकवला. पण एक कडू सत्य उरलंय, जे पुढच्या दोन chapters चा विषय आहे: **हे सगळं जाळं गुंतागुंतीचं झालंय, आणि गुंतागुंतीच्या जाळ्यात काहीतरी बिघडणारच.** मोठं system कधीच “पूर्ण चालू” नसतं; त्याचे तुकडे सतत पडत-उठत असतात. ते गृहीत धरून बांधणं: हीच पुढची, आणि सर्वात प्रौढ, नजर.

महाग चूक

”CDN म्हणजे फक्त मोठ्या video-company साठी, आमच्या साध्या app ला कशाला?” आजच्या जगात कुठलंही app जगभर वापरलं जाऊ शकतं, आणि images/design हे प्रत्येक app मध्ये असतंच: CDN शिवाय दूरचा गिन्हाईक प्रत्येक फोटोला अंतराचा कर भरतो, app रेंगाळतं, तो सोडून जातो. उलट चूकही आहे: **बदलणारी वैयक्तिक माहिती CDN वर टाकणं** (रमेशला सुनीलची जुनी शिल्लक दिसणं): CDN फक्त static साठी. “काय जवळ, काय मुळाशी” ही रेषा न ओळखणं म्हणजे एकतर हळू app किंवा चुकीची माहिती.

नकाशावर

1.1-1.4 यंत्र | ACID | index | cache
 1.5-1.8 replication | sharding | queue | load balancer
 1.9 CDN = गिन्हाईकाजवळ प्रती (cache चं भौगोलिक रूप);
 static जवळ, dynamic मुळाशी; वेग विकत, ताजेपणा विकून

सूत्रावर: CDN ने आरतीने जगभर वेग विकत घेतला, ताजेपणाची थोडी किंमत देऊन: पण फक्त static साठी. विभागभर एकच धडा गिरवला: **प्रत्येक सोय एक सौदा; सौदा दिसणं हीच अभियांत्रिकीची नजर.**

स्वतः बघा (5 मिनिटं)

कुठलंही मोठं app/website उघडा: design, images भरभर येतात (CDN, जवळून), पण तुमची वैयक्तिक माहिती (तुमच्या ऑर्डर, संदेश) क्षणभर उशिरा (मुळापासून). हा सूक्ष्म फरक लक्षात घ्या: “सगळ्यांचं सारखं” जवळून, “फक्त तुमचं” दुरून. आता तुम्हाला मागचं जाळं दिसतंय.

विचार करा

आरतीचं app जगभर, वेगवान, टिकाऊ. पण एका सकाळी अचानक सगळंच ठप्प: server चालू, database चालू, पण एका छोट्या आतल्या सेवेने (payment-तपासणी) उत्तर द्यायचं थांबवलं, आणि तिची वाट बघत **बाकी सगळं** अडकून बसलं: एका दुव्याने पूर्ण साखळी थांबली. आरती हादरली: “प्रत्येक भाग तयार असूनही एक छोटा भाग अडला की सगळं का थांबतं? आणि मोठ्या company मध्येही असं होतं का?”

होतं, आणि जगातल्या सर्वात मोठ्या company नाही होतं: Facebook, Amazon, अख्खं internet काही तासांसाठी बंद पडलंय. मोठं system कधीच “पूर्ण चालू” नसतं: अपयश हेच त्याचं सामान्य रूप आहे. ते गृहीत धरून बांधण्याची कला: पुढची भिंत, आणि ती सर्वात महत्त्वाची.

Chapter 1.10 [SPINE]: CAP: तीनातले दोनच मिळतात

Replication पासून (1.5) एक प्रश्न घोंगावतोय: प्रती एकसारख्या कशा राहणार? आता त्याचं मूळ उत्तर: आणि ते एका कठोर, सुंदर नियमात आहे: **CAP theorem**. हा नियम एकदा समजला की distributed systems मधले अर्थ वाद संपतात.

तीन इच्छा. अनेक यंत्रांच्या (distributed) system कडून आपल्याला तीन गोष्टी हव्या असतात:

- C Consistency (सुसंगती): कुठल्याही यंत्राला विचारा, तेच, ताजं उत्तर. रमेशची ऑर्डर सगळ्या प्रतींवर लगेच सारखी.
- A Availability (उपलब्धता): विचारलं की उत्तर मिळतंच, कधीच "थांब, माहीत नाही" नाही.
- P Partition tolerance (फूट-सहनशीलता): दोन यंत्रांमधला संपर्क तुटला (network फुटली) तरी system चालूच राहते.

कठोर सत्य: तिन्ही एकदम मिळत नाहीत. आणि गंमत अशी की अनेक-यंत्रांच्या जगात **P (फूट) टाळताच येत नाही**: तारा तुटतात, network मध्ये अडथळे येतात: कधीतरी दोन यंत्रांचा संपर्क तुटणारच. म्हणून खरा निर्णय दोनातलाच उरतो: **फूट पडली त्या क्षणी, C निवडायचं की A?**

दोन कारकुनांची उपमा (हीच पूर्ण CAP आहे): दोन गावांतल्या दोन कारकुनांकडे एकाच खात्याची वही आहे, आणि त्यांच्यातला फोन बंद पडलाय (partition). आता गिन्हाईक एका कारकुनाकडे येऊन विचारतो "माझी शिल्लक?" कारकुनापुढे दोनच पर्याय:

- **A निवड (उपलब्धता):** "ही घ्या शिल्लक" (स्वतःच्या वहीतली): उत्तर मिळालं, पण दुसऱ्या गावात कदाचित बदल झाला असेल, म्हणजे हे उत्तर **कदाचित जुनं** (C सोडलं).
- **C निवड (सुसंगती):** "फोन येईपर्यंत थांबा, खात्री करून सांगतो": उत्तर **अचूकच** असेल, पण तोवर गिन्हाईक **ताटकळला** (A सोडलं).

एकच "बरोबर" उत्तर नाही: **माहितीच्या स्वभावावर** ठरतं. आणि इथे या पुस्तकाचं सूत्र शिखरावर: CAP म्हणजे "काय विकून काय घ्यायचं" चा सर्वात शुद्ध नियम.

कुठे काय: आरतीचे निर्णय. (1) **पैसे/शिल्लक: C निवडा.** "थोडं थांबा" चालेल, पण चुकीची शिल्लक नको (दुहेरी खर्च, कायदेशीर घोळ). बँका इथे नेहमी C: फूट पडली तर व्यवहार थांबवतात, चुकीचा आकडा दाखवत नाहीत. (2) **like-count / "किती जणांनी बघितलं" / दुकानाची जुनी यादी: A**

निवडा. उत्तर आलंच पाहिजे (गिन्हाईक थांबणार नाही); 5 सेकंद जुना like-count चालतो. Social media इथे नेहमी A. **एकाच app मध्ये, माहितीगणिक वेगळी निवड:** हीच प्रौढ रचना (replication chapter चा “ताज-हवं की जुन-चालेल” तोच धागा, आता नियमात बांधलेला).

महाग चूक

”आम्हाला तिन्ही हवेत: नेहमी ताजं, नेहमी उपलब्ध, आणि फूट सहन करणारं.” हे मागणं म्हणजे भौतिकशास्त्राशी भांडण: तिन्ही एकदम अशक्य आहेत (सिद्ध झालेलं). जो engineer “तिन्ही देतो” असं म्हणतो तो एकतर खोटं बोलतोय किंवा त्याला अजून फूट भेटलेली नाही (आणि ती भेटणारच). नवखा तिन्हीची अपेक्षा करतो; जाणकार **आधीच ठरवतो**: “या माहितीला फूट पडली तर मी C निवडेन की A”: आणि तो निर्णय माहितीच्या स्वभावावर, कोड लिहिण्याआधी.

नकाशावर

- 1.1-1.9 यंत्र...CDN (नऊ भिंती, नऊ सौदे)
 1.10 CAP: फूट अटळ; मग C की A? तीनातले दोनच; पैसे->C
 (थांबा पण अचूक), like->A (जुनं पण उत्तर मिळो)

सूत्रावर: CAP हा सूत्राचा गाभा-नियम: distributed जगात “सगळं एकदम” असं काहीच नसतं; प्रत्येक क्षणी एक विकून एक घ्यावंच लागतं. ही निवड टाळता येत नाही; फक्त **जाणून** किंवा **नकळत** करता येते. जाणकार जाणून करतो.

स्वतः बघा (5 मिनिटं)

दोन अनुभव आठवा: (1) बँकेत/UPI त कधी “system busy, थोड्या वेळाने प्रयत्न करा” आलंय? ती बँकेची **C निवड** होती: अचूकतेसाठी उपलब्धता सोडली. (2) Social media वर like-count कधी मागे-पुढे झालेला दिसला? ती **A निवड**: उत्तर देत राहायला अचूकता सोडली. एकच तत्त्व, दोन विरुद्ध निर्णय: माहितीच्या स्वभावाने.

विचार करा

आरतीला आता एक विचित्र गोष्ट जाणवली: तिच्या मोठ्या system मध्ये छोटे-छोटे बिघाड **सतत** होत असतात: एखादा server क्षणभर अडतो, एखादी सेवा उशिरा उत्तर देते: पण गिन्हाईकाला कळतही नाही. पण कधीकधी एक छोटा बिघाड **पसरतो** आणि सगळं कोसळतं (1.9 चा payment-अडथळा). आरती

विचारते: “छोटे बिघाड तर होणारच. मग ते पसरून सर्वनाश होऊ नये, एका जागी थांबावेत: याची रचना कशी करायची?”

हाच distributed systems मधला सर्वात प्रौढ प्रश्न: **अपयश गृहीत धरून, ते मर्यादित ठेवणारी रचना.** Timeouts, retries, आणि “circuit breaker”: एका बिघाडाला साखळी होऊ न देणारी यंत्रणा. पुढची भिंत.

Chapter 1.11 [SPINE]: अपयश हेच सामान्य: Timeouts, Retries, Circuit Breakers

आरतीचा प्रश्न distributed systems च्या हृदयाला हात घालतो. आणि उत्तराआधी एक मनोवृत्ती-बदल: मोठं system कधीच “पूर्ण चालू” नसतं. त्याचे शेकडो तुकडे सतत पडत-उठत असतात. नवखा “सगळं चालू आहे” गृहीत धरून बांधतो; जाणकार “काहीतरी बिघडलेलंच असणार” गृहीत धरून बांधतो. हा फरक सगळं ठरवतो.

समस्या: एक अडला की साखळी अडते. आरतीच्या 1.9 मधला प्रसंग: payment-सेवा अडली, आणि तिची वाट बघत ऑर्डर-सेवा अडली, आणि तिची वाट बघत app अडलं. एका दुकानासमोर रांग लागली म्हणून अख्खा बाजार ठप्प. याला **cascading failure** (साखळी-कोसळ) म्हणतात, आणि तीन साध्या युक्त्या ती थांबवतात:

युक्ती 1: Timeout (वाट किती बघायची ठरवा). ऑर्डर-सेवा payment-सेवेला विचारते आणि अनंत काळ वाट बघते: हीच मूळ चूक. उपाय: “2 सेकंदांत उत्तर नाही आलं तर वाट सोडून द्या.” हॉटेलात वेटरला “5 मिनिटांत नाही आलं तर सांगा, दुसरं मागवतो” म्हणण्यासारखं. Timeout शिवाय एक हळू सेवा पूर्ण system ला गोठवते. नियम: **प्रत्येक बाहेरच्या विनंतीला timeout: वाट अनंत कधीच नाही.**

युक्ती 2: Retry (पुन्हा प्रयत्न: पण जपून). क्षणिक बिघाड (network चा हिचकी) असेल तर पुन्हा विचारणं शहाणपण. पण इथे सापळा: सेवा आधीच भाराने कोसळतेय आणि सगळे एकदम पुन्हा-पुन्हा विचारू लागले तर ती अजून बुडते (आगीत तेल). म्हणून: retry थांबत-थांबत (आधी 1 सेकंद थांब, मग 2, मग 4: “backoff”), आणि मर्यादित (3 वेळा, मग सोडून द्या). आणि retry सोबत idempotency हवीच (1.7: पुन्हा विनंती = दुहेरी काम नको). नियम: **retry करा, पण थांबत आणि मोजून.**

युक्ती 3: Circuit Breaker (वीज-fuse चं तत्त्व). घरातला fuse आठवा: short-circuit झाला की fuse उडतो आणि पूर्ण घर वाचतो: एका बिघाडाला आग होऊ देत नाही. Circuit breaker तेच: payment-सेवा वारंवार फसतेय हे दिसलं की, ऑर्डर-सेवा तिला विचारणं थांबवते काही काळ (“circuit उघडला”): “ती सेवा आजारी आहे, तिला त्रास देऊन दोघेही बुडायला नको.” तेवढ्या वेळात ती सेवा सावरते; मग हळूच एक-दोन विनंत्या सोडून “बरी झाली का” तपासते. एका आजारी दुव्याला वेगळं काढून बाकी system वाचवणं: हीच circuit breaker ची कला.

तिन्हींच एक तत्त्व: एका भागाचं अपयश दुसऱ्यात पसरू देऊ नका. प्रत्येक दुवा गृहीत धरतो की समोरचा कधीही फसू शकतो, आणि त्यासाठी आधीच तयार असतो: timeout ने वाट मर्यादित, retry ने क्षणिक बिघाड सावरतो, circuit breaker ने आजारी दुवा वेगळा करतो. हे “अपयश गृहीत धरून बांधणं”

(design for failure) म्हणजे मोठ्या systems ची सर्वात प्रौढ सवय: Book 4 च्या brakes चा (4.5) हा distributed भाऊ.

महाग चूक

”सगळं नीट चालेल असं गृहीत धरून बांधू; बिघडलं तर बघू.” हीच नवख्याची मूळ चूक: “happy path” (सगळं-सुरळीत रस्ता) बांधणं आणि अपयशाची तयारी न करणं. मग खरा भार आला की एक छोटा हिचकी साखळी होऊन सर्वनाश (1.9 चा प्रसंग). Timeout नाही म्हणून गोठतं, retry बेभान म्हणून आग भडकते, circuit breaker नाही म्हणून आजारी दुवा सगळ्यांना बुडवतो. जाणकार उलट बांधतो: **आधी विचारतो “हे फसलं तर?”, मग बांधतो.** “फसलं तर” चा विचार न करणारं प्रत्येक मोठं system शेवटी कोसळतंच: फक्त वेळ ठरलेली नसते.

नकाशावर

1.1-1.10 यंत्र...CAP

1.11 अपयश सामान्य: timeout (वाट मर्यादित) + retry (थांबत, मोजून) + circuit breaker (आजारी दुवा वेगळा); एका भागाचं अपयश पसरू देऊ नका

सूत्रावर: इथे सूत्र मनोवृत्तीत उतरतं: टिकाव विकत घ्यायचा तर “सगळं चालेल” चा भाबडेपणा विकावा लागतो. जो अपयश गृहीत धरतो तोच टिकणारं बांधतो.

स्वतः बघा (5 मिनिटं)

घरातला fuse/MCB बघा: तो का असतो? एका बिघाडाला (short- circuit) पूर्ण घर जळू न देण्यासाठी: तो मुद्दाम “तुटतो” म्हणून घर वाचतं. Software चा circuit breaker अगदी हेच करतो. आणि app वापरताना “काहीतरी चुकलं, पुन्हा प्रयत्न करा” दिसलं तर ओळखा: मागे timeout/retry ची यंत्रणा तुम्हाला अनंत-गोठण्यापासून वाचवतेय.

विचार करा

आरतीच्या app मध्ये हे सगळे बचाव आहेत. पण एका रात्री गिन्हाईकांच्या तक्रारी येऊ लागल्या “app हळू आहे”: आणि आरतीला **कळेचना कुठे** हळू आहे. वीस servers, अनेक सेवा, database, cache, queue: बिघाड नक्की कुठे? ती अंधारात चाचपडत होती. “एवढं मोठं जाळं आहे; आत काय चाललंय हे मला **दिसलंच** नाही तर दुरुस्त कसं करणार?”

हाच मोठ्या systems चा अदृश्य पाया: **दिसणं**. आत काय चाललंय हे मोजता-बघता आलं नाही तर सगळे बचाव आंधळे. प्रत्येक विनंतीचा माग, प्रत्येक सेवेची तब्येत, आणि “टोक” मोजणं: याचं नाव observability. पुढची भिंत (Book 4, 5.4 ची इथे भेट).

Chapter 1.12 [SPINE]: दिसणं: Observability आणि p99 ची गोष्ट

आरतीचा “आत काय चाललंय दिसलंच नाही” हा पेच मोठ्या systems चा अदृश्य पाया आहे. Book 4 त हेच AI-agent साठी भेटलं (5.4); इथे ते पूर्ण system साठी, आणि एका फसव्या आकड्याची गोष्ट घेऊन.

तीन डोळे (Book 4, 5.4 चे तेच, आता system-भर). (1) **Logs** (नोंदी): प्रत्येक घटनेची ओळ: “2:03 ला रमेशची ऑर्डर आली, payment अडलं.” (2) **Metrics** (आकडे): “आता किती विनंत्या/सेकंद, किती फसल्या, किती वेळ लागतोय.” (3) **Traces** (माग): एका विनंतीचा पूर्ण प्रवास: load balancer -> ऑर्डर-सेवा -> database -> payment: कुठे किती वेळ. आरतीच्या “कुठे हळू” चं उत्तर trace मध्ये: तिला दिसलं की payment-सेवा प्रत्येक विनंतीला 3 सेकंद खातेय. अंधारात चाचपडणं संपलं: **दिसलं, म्हणून दुरुस्त झालं.**

आणि आता तो फसवा आकडा: सरासरी. आरतीचं dashboard म्हणत होतं: “सरासरी उत्तर-वेळ: 0.4 सेकंद”: उत्तम! मग तक्रारी का? कारण **सरासरी खोटं बोलते.** कल्पना करा: 100 गिन्हाईकांपैकी 95 जणांना 0.1 सेकंदात उत्तर, पण 5 जणांना 8 सेकंद. सरासरी? ~0.5 सेकंद: गोड दिसतं. पण **तो प्रत्येक विसावा गिन्हाईक 8 सेकंद ताटकळतोय** आणि रागाने app सोडतोय. सरासरी त्या 5 जणांना लपवते.

म्हणून खरं मोजमाप: टोक (percentiles). सरासरीऐवजी अनुभवी teams **p95, p99** बघतात: “95% (किंवा 99%) गिन्हाईकांना या-पेक्षा-जलद उत्तर मिळतं; उरलेलं टोक किती वाईट?” आरतीचा p99 होता **8 सेकंद:** म्हणजे प्रत्येक शंभरातला एक जण भयंकर ताटकळत होता. एका मोठ्या app मध्ये “शंभरातला एक” म्हणजे रोज हजारो नाराज गिन्हाईक. आणि इथे एक कठोर सत्य: **धंदा सरासरीने मोजला जात नाही, टोकाने मोजला जातो.** सर्वात वाईट अनुभव घेणारा गिन्हाईकच सर्वात मोठ्याने बोलतो, सोडून जातो, वाईट review देतो. म्हणून मोठ्या systems मध्ये p99 (कधी p99.9) हाच खरा आरसा; सरासरी हा भूल देणारा आरसा.

Observability चं तत्त्व: न दिसणारं दुरुस्त करता येत नाही; आणि चुकीचं मोजणारं चुकीचं दुरुस्त करतं. आरतीने सरासरीवर विसंबून राहिली असती तर “सगळं ठीक आहे” म्हणत बसली असती, गिन्हाईकं गळत असताना. p99 बघितलं म्हणून खरी अडचण दिसली, payment-सेवा दुरुस्त झाली, टोक 8 सेकंदांवरून 0.5 वर आलं. दिसणं + योग्य मोजमाप = मोठ्या system चं आरोग्य.

महाग चूक

“सरासरी छान आहे, system निरोगी.” मोठ्या systems मधली सर्वात सामान्य आत्म-फसवणूक. सरासरी नेहमी गोड दिसते कारण ती बहुसंख्य- सुखी लोकांत अल्पसंख्य-दुःखी लपवते: पण तेच दुःखी लोक तुमचा धंदा ठरवतात (ते बोलतात, सोडतात, review देतात). सरासरीवर system “निरोगी” म्हणणं म्हणजे “गावात सरासरी उत्पन्न चांगलं आहे” म्हणत गरिबी नाकारणं. नेहमी टोक (p95/p99) बघा: तिथे खरं दुखणं राहतं.

नकाशावर

1.1-1.11 यंत्र...अपयश-बचाव

1.12 observability: logs + metrics + traces = दिसणं; सरासरी खोट बोलते, टोक (p99) खरं; न दिसणारं दुरुस्त होत नाही

सूत्रावर: दिसणं हे फुकट नाही (नोंदी-आकडे साठवायचा खर्च: Book 4, 5.4): पण न-दिसण्याची किंमत त्याहून प्रचंड (आंधळं दुरुस्तीकाम, गळणारे गिऱ्हाईक). दृश्यता विकत घेणं हा शहाणा सौदा.

स्वतः बघा (5 मिनिटं)

एक विचार-प्रयोग: तुमच्या गावातल्या 10 लोकांचं उत्पन्न: 9 जणांचं Rs 20,000, एकाचं Rs 10 लाख. सरासरी? ~1.2 लाख: “गाव श्रीमंत”! पण 9 जण गरीब. हीच सरासरीची फसवणूक. आता उलट: 9 जणांना app 0.1 सेकंदात उघडतं, एकाला 8 सेकंद: सरासरी “वेगवान” म्हणेल, पण तो एक जण संतापलेला. टोक बघायला शिका: आयुष्यातही, system मध्येही.

विचार करा

आरतीने सगळं शिकलं: पण एका सकाळी उठून बघते तर तिचं app नाही, **अर्ध internet** बंद: तिचे servers चालू, code चालू, पण ज्या मोठ्या cloud वर ती अवलंबून होती त्यातच बिघाड. तिच्यासारख्या हजारो companies एकदम ठप्प. आरती सुत्र: “एवढ्या मोठ्या, अब्जावधींच्या company सुद्धा कोसळतात? आणि एका छोट्या चुकीने अख्खं internet? हे कसं शक्य आहे?”

शक्य आहे, आणि घडलंय: वारंवार. सर्वात मोठ्या company नी सर्वात मोठे अपघात केलेत, आणि प्रत्येक अपघातात एक अनमोल धडा आहे. पुढचा chapter शव-विच्छेदन आहे: ज्या दिवशी जग बंद पडलं.

Chapter 1.13 [SPINE]: शव-विच्छेदन: ज्या दिवशी जग बंद पडलं

हा chapter उलटा: चार खऱ्या महाप्रलयांची शव-विच्छेदनं (तपासलेली). हेतू भीती नाही; उलट दिलासा आणि धडा: जगातल्या सर्वात हुशार teams नीही हे भोगलंय, म्हणजे अपयश लाज नाही: न शिकणं लाज आहे.

प्रलय 1: Amazon S3, Feb 2017: एका टंकलेखन-चुकीने. Amazon चा S3 (जगाचा अर्धा data जिथे राहतो ती साठवण-सेवा) चार तास बंद पडली. कारण? एक अधिकृत engineer, ठरलेल्या पद्धतीने billing-यंत्रणा दुरुस्त करताना, एका command मध्ये चुकीचा आकडा टाकला: ठरवलेल्यापेक्षा कितीतरी जास्त servers बंद झाले. आणि गंमत (कडू): Amazon चं “सगळं व्यवस्थित आहे का” दाखवणारं dashboard स्वतः S3 वरच चालत होतं: म्हणून S3 बंद पडल्यावर dashboard ही लाल दाखवू शकेना: Amazon ला Twitter वरून सांगावं लागलं! धडा: एका माणसाची एक टंकचूक जग बंद करू शकते: म्हणून धोक्याच्या commands ना रचनात्मक अडसर हवेत (Book 4, 4.5 चे brakes; एकाच command ने प्रचंड विध्वंस अशक्य करणं).

प्रलय 2: Knight Capital, 2012: 45 मिनिटांत \$460 million. ही सर्वात थरारक. एका trading-company ने नवें code आठ servers वर टाकायचं होतं; engineer एक server विसरला. त्या एका जुन्या server वर एक मृत, विसरलेलं जुनं code जागं झालं आणि वेड्यासारखे चुकीचे शेअर-व्यवहार करू लागलं. 45 मिनिटांत कंपनीने ~\$460 million गमावले: आणि कंपनीच बुडाली. सकाळी 8 वाजता 97 इशारा-emails आले होते (“जुनं code जागं झालंय”): कुणीही वाचले नाहीत. धडा दुहेरी: अर्धवट deployment (एक server विसरणं) = मृत्यू (म्हणून पुढे 3.7 त canary/सुरक्षित-deploy); आणि इशारे वाजले पण कुणी बघत नव्हतं = observability असून पाहणारा नसेल तर व्यर्थ (1.12 चा कडू पूरक).

प्रलय 3: Facebook, Oct 2021: 6 तास, आणि दारंही उघडेनात. एका नेहमीच्या देखभाल-command ने चुकून Facebook च्या सगळ्या data-centers चा जगाशी संपर्क तोडला (BGP/DNS: internet चा पत्ता-यंत्रणा). Facebook, WhatsApp, Instagram: 6 तास गायब. आणि सर्वात विचित्र: engineers ना दुरुस्त करायला data-center मध्ये शिरताच येईना, कारण दाराचे electronic बॅज-lock सुद्धा त्याच network वर होते जे बंद पडलं होतं! धडा: तुमची दुरुस्ती-यंत्रणा त्याच गोष्टीवर अवलंबून असेल जी बिघडली, तर तुम्ही अडकता (circular dependency): बचाव-यंत्रणा नेहमी वेगळी, स्वतंत्र हवी.

प्रलय 4: CrowdStrike, Jul 2024: 85 लाख यंत्रं, एका update ने. एका सुरक्षा-company ने एक सदोष update पाठवलं आणि जगभरची ~85 लाख Windows यंत्रं एकदम कोसळली: विमानतळ, रुग्णालयं, बँका ठप्प (इतिहासातला सर्वात मोठा IT-बिघाड, यंत्र-संख्येने). कारण: update **सगळ्यांना एकदम गेलं**, आधी थोड्यांवर तपासून नाही. धडा: **कुठलाही बदल सगळ्यांना एकदम देऊ नका; आधी थोड्यांवर** (हाच 3.7 चा canary; इथे त्याची गैरहजेरी अब्जांची पडली).

चार प्रलयांचं समान सूत्र: प्रत्येक ठिकाणी बिघाड **लहान** होता (एक टंकचूक, एक विसरलेला server, एक command, एक update); तो **पसरला** म्हणून महाप्रलय. म्हणजे 1.11 चा धडा जगाच्या पातळीवर: **अपयश टाळता येत नाही; पसरणं टाळता येतं.** आणि दुसरा, दिलासा देणारा धडा: या सगळ्या company नी नंतर **प्रामाणिक शव-विच्छेदनं जाहीर** केली (कुणाला दोष न देता, “रचना कशी बदलली”): तीच प्रौढ संस्कृती (3.6 त पूर्ण).

महाग चूक

”आमचं छोटं app; असे महाप्रलय आम्हाला नाहीत.” उलट: या company कडे जगातले सर्वोत्तम engineers, अब्जांची यंत्रणा, तरी कोसळल्या: कारण अपयश **कौशल्याचा** प्रश्न नाही, **गुंतागुंतीचा** अपरिहार्य साथीदार आहे. तुमचं छोटं app कोसळलं तर बातमी होणार नाही, पण गिऱ्हाईक जाईल तोच. धडा उलट घ्या: मोठेही कोसळतात, म्हणून **अपयश गृहीत धरून बांधणं** ही लाज नाही, शहाणपण आहे: आणि कोसळल्यावर **दोष न देता शिकणं** ही खरी प्रौढता.

नकाशावर

1.1-1.12 यंत्र...observability
1.13 चार प्रलय: S3 (टंकचूक) | Knight (विसरला server, \$460M)
| FB (दारंही बंद) | CrowdStrike (85 लाख, एकदम update);
बिघाड लहान, पसरला म्हणून प्रलय; canary + स्वतंत्र बचाव

सूत्रावर: गुंतागुंत विकत घेतली की अपयश-शक्यता फुकट मिळते: मोठेपणाची न-टाळता-येणारी किंमत. शहाणपण अपयश टाळण्यात नाही (अशक्य), **पसरू न देण्यात आणि शिकण्यात** आहे.

स्वतः बघा (5 मिनिटं)

कधी असा दिवस आठवा जेव्हा WhatsApp/Instagram/UPI काही तास बंद होतं (2021 पासून अनेकदा घडलंय). तेव्हा वाटलं होतं “एवढी मोठी company, कशी बंद पडली?": आता उत्तर माहीत आहे: गुंतागुंतीच्या जाळ्यात एक लहान बिघाड पसरला. मोठेपणा आणि नाजूकपणा एकत्रच येतात: हेच या

chapter चं सत्य.

विचार करा

आरतीचं app आता स्थिरावलंय: वेगवान, टिकाऊ, दृश्य, अपयश- सज्ज. पण महिना-अखेरीस cloud चं bill आलं आणि आरती हादरली: अपेक्षेच्या तिप्पट! सगळं “चालतंय” पण खर्च गिळंकृत करतोय. ती विचारते: “servers, database, CDN, हे सगळं तर लागतंच. मग bill एवढं का, आणि नक्की कशाचं?”

मोठ्या systems मध्ये bill हे code इतकंच रचनेचं फळ असतं: आणि त्यात एक लपलेला, कपटी खर्च असतो जो बहुतेकांना उशिरा कळतो. Cloud-bill चं शरीर: विभागाची शेवटची भिंत.

Chapter 1.14 [DEPTH]: Cloud-Bill चं शरीर आणि Egress-सापळा

आरतीच्या तिप्पट-bill च्या धक्क्यातून एक व्यावहारिक chapter. [DEPTH]: घाई असेल तर “egress-सापळा” आणि महाग-चूक वाचा; पण थांबलात तर तुमचं भविष्यातलं bill वाचेल.

Cloud चं भाडं कशाचं येतं: चार खण. Book 1 ने cloud दिलं (भाड्याने यंत्रं). त्या भाड्याचे चार मुख्य भाग: (1) **गणित** (compute: servers चालू ठेवणं: तास-गणिक); (2) **साठवण** (storage: database, फोटो, backups: GB-गणिक); (3) **वाहतूक** (data ची ने-आण); (4) **खास सेवा** (तयार database, queue, वगैरे: सोयीचे, पण महाग). बहुतेकांना पहिले दोन माहीत असतात; आणि नेमका तिसरा त्यांना बुडवतो.

Egress-सापळा (सर्वात कपटी खर्च). Cloud चा एक विचित्र नियम: **data आत आणणं (ingress) बहुधा फुकट; data बाहेर काढणं (egress) महाग.** माशाच्या गळाची उपमा: आत ओढणं सोपं, फुकट; बाहेर पडणं महाग. आरतीचं bill का फुगलं? तिचे फोटो एका cloud मध्ये, app दुसरीकडे: प्रत्येक गिन्हाईकाला फोटो दाखवायला data **एका cloud मधून बाहेर** पडत होता: लाखो वेळा, प्रत्येक वेळी egress-भाडं. आणि हा खर्च **वापर वाढला की वाढतो**, म्हणून सुरुवातीला दिसत नाही, यश आलं की अचानक चावतो. CDN (1.9) egress कमी करतं (जवळून देतं, मुख्य cloud मधून वारंवार बाहेर नको): म्हणजे CDN फक्त वेगासाठी नाही, **bill साठीही**.

आणि सर्वात मोठा दीर्घकालीन सापळा: lock-in. Egress महाग ठेवण्यामागचा cloud चा हेतू उघड आहे: **तुमचा data आत ओढून घ्यायचा, बाहेर काढणं महाग करून तुम्हाला अडकवायचं.** एकदा अब्जावधी फोटो एका cloud मध्ये साठले की दुसरीकडे हलवायला egress-भाडंच इतकं की तुम्ही अडकता. हा जाणीवपूर्वक सौदा आहे: cloud सोय देतं, बदल्यात तुम्हाला बांधून ठेवतं. जाणकार हे **आधी** ओळखतो आणि रचना अशी करतो की गरज पडली तर हलता येईल (Book 4, 2.8 चा खोगीर-नियम इथेही: एका जागी बांधून पडू नका).

सूत्र इथे रोखठोक: cloud = सोय + वेग विकत, पैसा + स्वातंत्र्य विकून. सुरुवातीला सोय प्रचंड (यंत्रं विकत घ्यायची नाहीत, मिनिटांत सुरू); पण वाढलं की egress, खास-सेवांचं भाडं, lock-in: किंमत चढते. म्हणून Book 4 चा तोच निर्णय (6.4): **पायाभूत सोयी भाड्याने, पण bill रोज मोजा आणि हलता येईल अशी रचना ठेवा.**

महाग चूक

”Cloud आहे, यंत्रं आपोआप वाढतात (auto-scale), निश्चित.” Auto-scaling सोयीचं, पण तेच bill-राक्षसाचं दार: भार वाढला की यंत्रं आपोआप वाढतात **आणि bill ही**: आणि कुणी लक्ष ठेवत नसेल तर एका चुकीच्या loop ने किंवा एका हल्ल्याने रातोरात लाखोंचं bill (खऱ्या घटना आहेत). नियम: **bill हे metric आहे, रोज बघा** (observability फक्त वेगासाठी नाही, खर्चासाठीही: 1.12 चा पूरक); आणि खर्चाला **मर्यादा** (alerts, budgets) लावा. न मोजलेलं cloud-bill हे न मोजलेल्या p99 इतकंच घातक.

नकाशावर

1.1-1.13 यंत्र...चार प्रलय

1.14 cloud-bill: गणित + साठवण + वाहतूक + खास-सेवा;
egress-सापळा (बाहेर काढणं महाग) + lock-in; bill हे
रोज बघायचं metric

सूत्रावर: cloud चा सौदा वेळेनुसार उलटतो: सुरुवातीला सोय स्वस्त, यशानंतर स्वातंत्र्य महाग. सौदा **कालानुरूप** बदलतो हे ओळखणं ही सर्वात सूक्ष्म नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या फोनवरच्या cloud-storage (Google Drive/iCloud) चा विचार करा: data **आत** टाकणं सोपं, फुकट-वाटतं; पण जागा भरली की पैसे, आणि सगळं दुसरीकडे हलवायचं म्हटलं की किती किचकट: हाच lock-in, ग्राहक-रूपात. मोठ्या companies तेच, egress-भाड्यासह अनुभवतात. सोयीमागचा सापळा एकदा दिसला की तुम्ही सावध खरेदीदार होता.

विचार करा

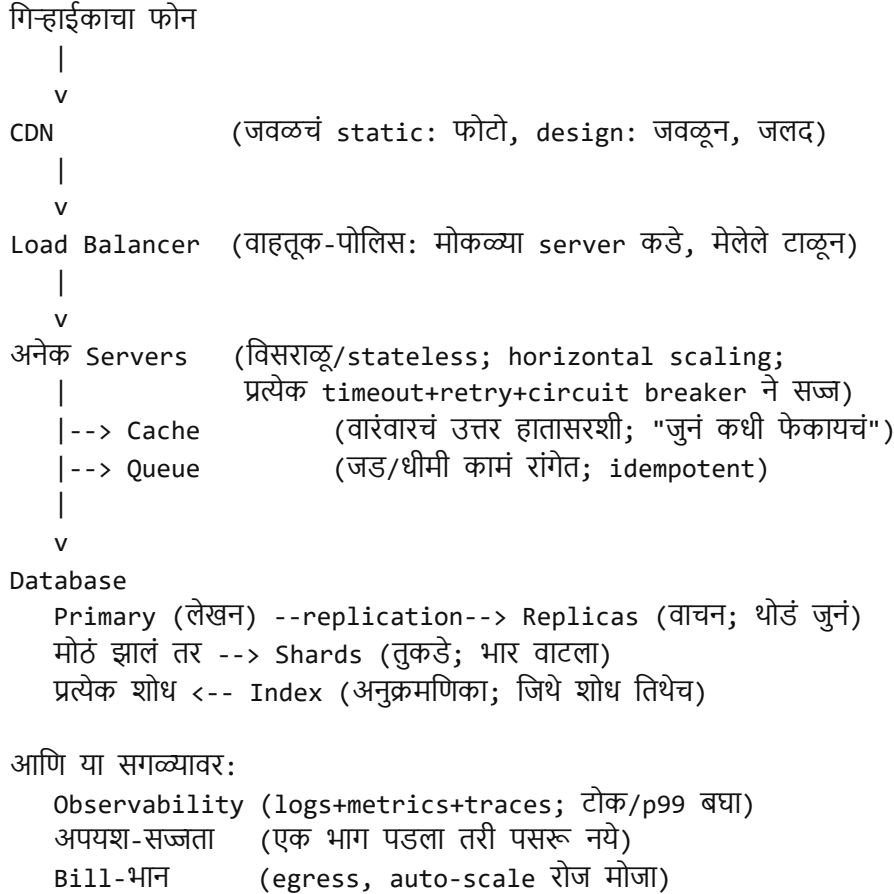
आरतीचा प्रवास संपत आलाय: एक laptop पासून जगभर पसरलेल्या, वेगवान, टिकाऊ, दृश्य, खर्च-जागरूक system पर्यंत. पण तिला एक गोष्ट खटकतेय: तिने हे सगळं **तुकड्या-तुकड्याने, भिंतीमागून भिंत** शिकलं. “आता हे सगळे तुकडे मिळून एक पूर्ण चित्र कसं दिसतं? आणि पुढच्या वेळी नवं app बांधताना हे सगळं **आधीच** कसं आखायचं, प्रत्येक भिंतीला आपटल्याशिवाय?”

हाच विभागाचा कळस: सगळे तुकडे एका चित्रात जोडणं, आणि “भिंत येण्याआधी दिसणं” ही नजर कमावणं. आरतीचं पूर्ण चित्र: विभागाचा शेवटचा chapter.

Chapter 1.15 [SPINE]: आरतीचं पूर्ण चित्र

विभागाचा कळस: सगळे तुकडे एका चित्रात. आरती मागे वळून बघते: एक laptop पासून इथवरचा प्रवास, आणि प्रत्येक भिंतीचा सौदा. हे चित्र पाठ करा; कुठलंही मोठं app याच आकाराचं असतं.

एका विनंतीचा आजचा प्रवास (सुरुवातीच्या सहा पायऱ्या, आता पूर्ण वाढलेल्या):



नऊ सौद्यांचा सारांश (हीच खरी शिकवण):

एक-यंत्र vs अनेक	सोपेपणा <-> वाढ+टिकाव
ACID	वेग <-> खात्री
index	लेखन+जागा <-> वाचन-वेग
cache / CDN	ताजेपणा <-> वेग
replication	ताजेपणाची-खात्री <-> टिकाव+वाचन-वेग
sharding	सोपे-प्रश्न <-> लेखन-वाढ
queue	"लगेच झालं" <-> लवचिकता (+idempotency)
CAP (फूट-क्षणी)	C (अचूक-पण-थांबा) <-> A (उत्तर-पण-जुनं)
cloud	पैसा+स्वातंत्र्य <-> सोय+वेग

आणि “भिंत येण्याआधी दिसणं” ची नजर (आरतीचा खरा प्रश्न): आता आरती नवं app बांधताना प्रत्येक भिंतीला आपटायची वाट बघत नाही; ती **आधीच** विचारते: “ही माहिती वाढणार का (index)? वारंवार तेच विचारलं जाणार का (cache)? एक यंत्र पडलं तर (replication+load balancer)? कुठलं काम धीमं आहे (queue)? फूट पडली तर C की A?” **पण: एकही गोष्ट गरजेआधी बांधायची नाही.** नजर म्हणजे “भिंत ओळखणं”, घाई म्हणजे “भिंत नसताना भिंत बांधणं” (premature scaling: 1.1, 1.6 चा इशारा). प्रौढ engineer भिंत **येताना दिसते** पण **आल्यावरच** बांधतो: आधी नाही (उगाच गुंतागुंत), खूप उशिरा नाही (crash).

सर्वात मोठा धडा, एका ओळीत: आरतीने कुठलीही “जादूई भारी रचना” शिकली नाही; तिने **सौदे बघायला** शिकलं. “सगळ्यात चांगलं software” असं काही नसतं; फक्त “या गर्दीला, या माहितीला, या क्षणी योग्य” असतं. आणि उद्या गर्दी बदलली की योग्य रचनाही बदलेल. **रचना ही स्थिर गोष्ट नाही; ती सतत बदलणाऱ्या सौद्यांची जिवंत निवड आहे.** हीच Part 1 ची कमाई, आणि Part 2 चा पाया: कारण Part 2 त आपण हे सौदे **मोठ्या रचना-निर्णयांना** लावू: SQL की NoSQL, एक-मोठं-app की अनेक-लहान-सेवा.

महाग चूक

”आता मला सगळं येतं, पुढच्या app मध्ये हे सगळं आधीच टाकतो.” Part 1 ची शिकवण **उलटी** आहे: तुम्हाला सगळे **सौदे** कळले, म्हणजे आता तुम्ही प्रत्येक गोष्ट **गरजेपुरती** टाकाल. ज्ञानाचा उपयोग “सगळं आधीच बांधणं” नाही, तर **”कधी काय लागेल हे ओळखून, तेव्हाच बांधणं.”** नवशिक्याची चूक: काहीच न जाणणं. अर्धवट-शिकलेल्याची चूक: सगळं आधीच बांधणं. जाणकाराची नजर: भिंत ओळखणं, आणि तिची वाट बघणं.

नकाशावर

Part 1 पूर्ण: प्रवासाचा पूर्ण नकाशा
CDN -> LB -> stateless servers (+cache, +queue) -> DB
(primary/replicas/shards, +index); त्यावर observability +
अपयश-सज्जता + bill-भान. नऊ सौदे. नजर = भित आधी दिसणं,
पण गरजेआधी न बांधणं.

सूत्रावर: पूर्ण Part 1 एका वाक्यात: **मोठेपणा फुकट नसतो; प्रत्येक भिंतीवर काहीतरी विकून काहीतरी घ्यावं लागतं, आणि जाणकार तो सौदा जाणून करतो.** हेच अभियांत्रिकीचं सौंदर्य.

स्वतः बघा (5 मिनिटं)

तुमचं आवडतं मोठं app (Zomato, WhatsApp, कुठलंही) घ्या आणि वरचं “पूर्ण चित्र” त्याला लावून बघा: त्यात CDN असेल (images भरभर), load balancer (कधीच “एकच server पडला” दिसत नाही), cache (काही गोष्टी झटकन, refresh वर बदलतात), queue (“processing, थांबा”). तुम्ही आता app च्या मागचं जाळं “वाचू” शकता: हीच Part 1 ची कमाई.

विचार करा

Part 1 संपला. आरतीला आता एक मोठा निर्णय घ्यायचाय: तिचं अख्खं app एकाच मोठ्या code मध्ये आहे (सोपं होतं सुरुवातीला). आता 30 engineers काम करतायत आणि एकमेकांच्या पायात येतायत: एकाने बदल केला की दुसऱ्याचं मोडतं. “app चे तुकडे करावेत का? प्रत्येक टीमला स्वतःचा तुकडा? की एकत्रच बरं?” आणि दुसरा प्रश्न: “माझी काही माहिती ACID-कडक database मध्ये आहे, पण like-count सारखी सैल माहिती त्याच कडक जागी कशाला? वेगळी, सैल database वापरू का?”

हे दोन प्रश्न Part 2 चे दोन सर्वात मोठे वाद आहेत: monolith vs microservices, आणि SQL vs NoSQL. आणि दोन्हीचं उत्तर Part 1 च्या एकाच सूत्रात आहे: **सौदा बघा, fashion नको.** Part 2, पान पहिलं: रचनेचे मोठे निर्णय.

BOOK 5, PART 1 चा शेवट

जे तुमच्याकडे आता आहे

एक server	तीन कामं (एक-विचार-लक्षात ठेव); सोपं पण एकच अपयश-बिंदू; premature scaling टाळा
database/ACID	शिस्तीची वही: अखंड-सुसंगत-एकांत-टिकाऊ; खात्री विकत, वेग विकून; backup तपासलेला हवा
index	अनुक्रमणिका; वाचन जलद, लेखन+जागा विकून; जिथे शोध तिथेच (सगळीकडे नको)
cache	हातासरशी उत्तर; वेग विकत, ताजेपणा विकून; खरा प्रश्न "जुनं कधी फेकायचं"
replication	प्रती; टिकाव+वाचन-वेग विकत, ताजेपणा विकून; ताजं-हवं मुख्याकडे, जुनं-चालेल प्रतीकडे
sharding	तुकडे; लेखन-वाढ विकत, एकत्रित प्रश्न विकून; shard key = भार-भाकीत; शेवटचा उपाय
queue	रांग; धक्का शोषतो; "किमान एकदा" -> idempotency (दोनदा केलं तरी परिणाम एकदाच)
load balancer	वाहतूक-पोलिस; horizontal scaling (अनेक स्वस्त > एक महाग); अट: servers विसराळू
CDN	गिऱ्हाईकाजवळ static; वेग+bill दोन्ही; dynamic मुळाशी
CAP	फूट अटळ; मग C (अचूक-थांबा) की A (उत्तर-जुनं); पैसे->C, like->A
अपयश-सज्जता	timeout + retry(थांबत) + circuit breaker; एका भागाचं अपयश पसरू देऊ नका
observability	logs+metrics+traces; सरासरी खोटं, टोक(p99) खरं
चार प्रलय	S3/Knight/FB/CrowdStrike: बिघाड लहान, पसरला म्हणून प्रलय; canary + स्वतंत्र बचाव
cloud-bill	egress-सापळा + lock-in; bill रोज बघा

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. App हळू झालं. पहिलं निदान "मोठा server घेऊ" का नाही?

कारण 90% हळूपणा रचनेत असतो, ताकदीत नाही: गहाळ index, गहाळ cache, वाचनं मुख्यावर (replicas कडे वळवा). पैसे ओतण्याआधी नजर. (1.3, 1.4, 1.5)

2. Queue वापरता तर idempotency का हवीच?

queue ची हमी “किमान एकदा” = तेच काम दोनदा येऊ शकतं; idempotency शिवाय दुहेरी payment/ऑर्डर. (1.7)

3. पैशाच्या शिल्लकीला CAP मध्ये C की A, आणि का?

C (सुसंगती): फूट पडली तर “थांबा” चालेल, पण चुकीचा आकडा नाही. like-count ला उलट (A). (1.10)

4. System निरोगी आहे हे सरासरी-वेळेवरून का ठरवू नये?

सरासरी बहुसंख्य-सुखींत अल्पसंख्य-दुःखी लपवते; धंदा टोकाने (p99) मोजला जातो: तोच गिन्हाईक बोलतो-सोडतो. (1.12)

पुढच्या भागाची चाहूल

आरतीचे दोन मोठे निर्णय Part 2 उघडतात: **app चे तुकडे करावेत का** (monolith vs microservices) आणि **कडक database की सैल** (SQL vs NoSQL). Part 2 त हे मोठे रचना-वाद, data modeling, API-रचना, आणि search: सगळं Part 1 च्याच एका सूत्राने: **सौदा बघा, fashion नको**.

BOOK 5, PART 1 चा MINI-GLOSSARY

ACID	database ची शिस्त: अखंड-सुसंगत-एकांत-टिकाऊ (1.2)
cache	वारंवारचं उत्तर जलद जागी; ताजेपणा विकून (1.4)
CAP	फूट-क्षणी C की A; तीनातले दोनच (1.10)
CDN	गिन्हाईकाजवळ static ची प्रत (1.9)
circuit breaker	आजारी दुवा वेगळा काढणारा fuse (1.11)
egress	cloud मधून data बाहेर काढण्याचं (महाग) भाडं (1.14)
horizontal scale	अनेक स्वस्त यंत्र > एक महाग (1.8)
idempotency	दोनदा केलं तरी परिणाम एकदाच (1.7)
index	database ची अनुक्रमणिका; वाचन जलद (1.3)
load balancer	विनंत्या मोकळ्या server कडे वळवणारा (1.8)
p99	टोकाचं मोजमाप; सरासरीची फसवणूक टाळते (1.12)
partition	अनेक यंत्रांमधला संपर्क तुटणं (1.10)
queue	जड कामं रांगेत; धक्का शोषतो (1.7)
replication	database ची प्रत; lag = किती जुनी (1.5)
shard key	तुकडा कशाने पाडायचा; भार-भाकीत (1.6)
sharding	database चे यंत्रांवर तुकडे; लेखन वाटलं (1.6)
stateless	server काहीही स्वतःकडे साठवत नाही (1.8)
timeout	वाट किती बघायची, मर्यादा (1.11)
vertical scale	तेच यंत्र अधिक शक्तिशाली करणं (1.8)